Advanced Natural Language Processing

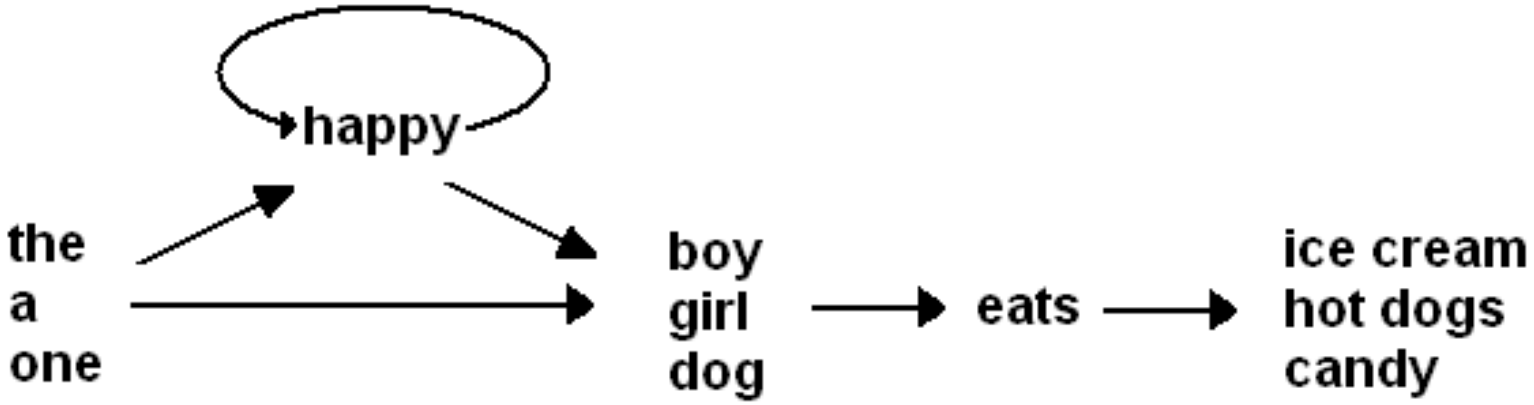
Statistical Approaches for NLP



Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027

Maastricht University

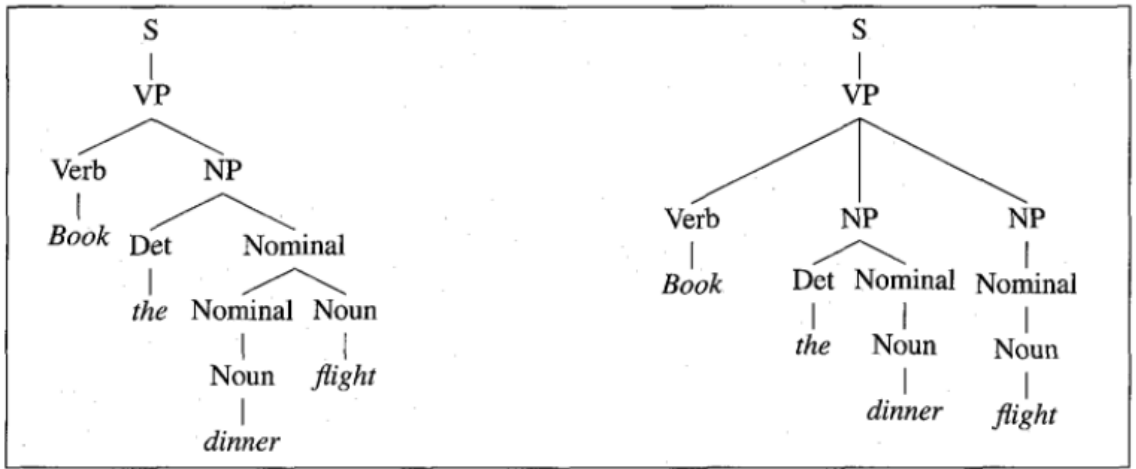
Last week: discrete Finite State Machines and Context-Free Grammars



Statistical parsing: transition probabilities

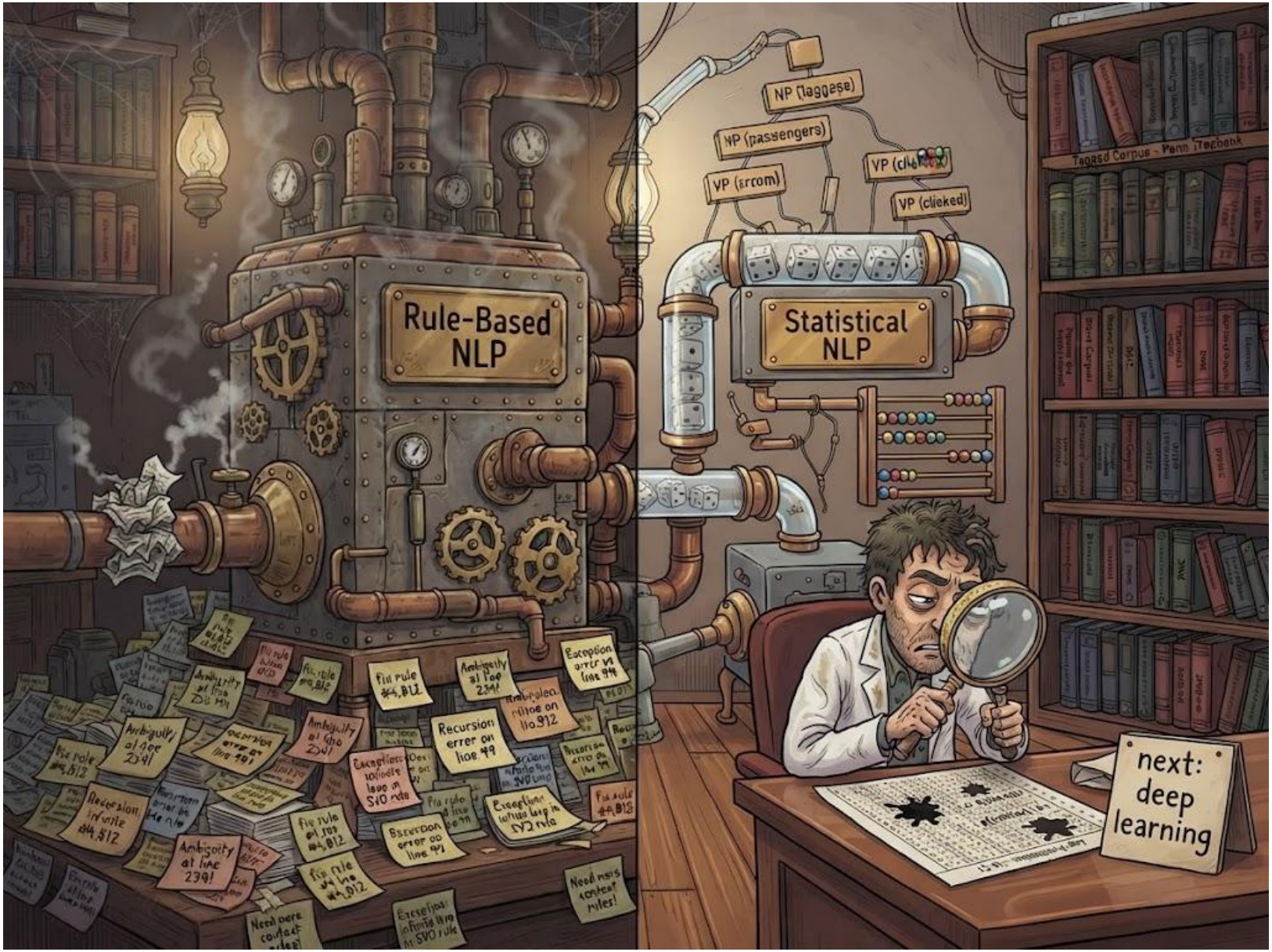
$$\sum_{\beta} P(A \rightarrow \beta) = 1$$

Disambiguating with statistics: probability of a particular syntactic structure or a generative grammar?

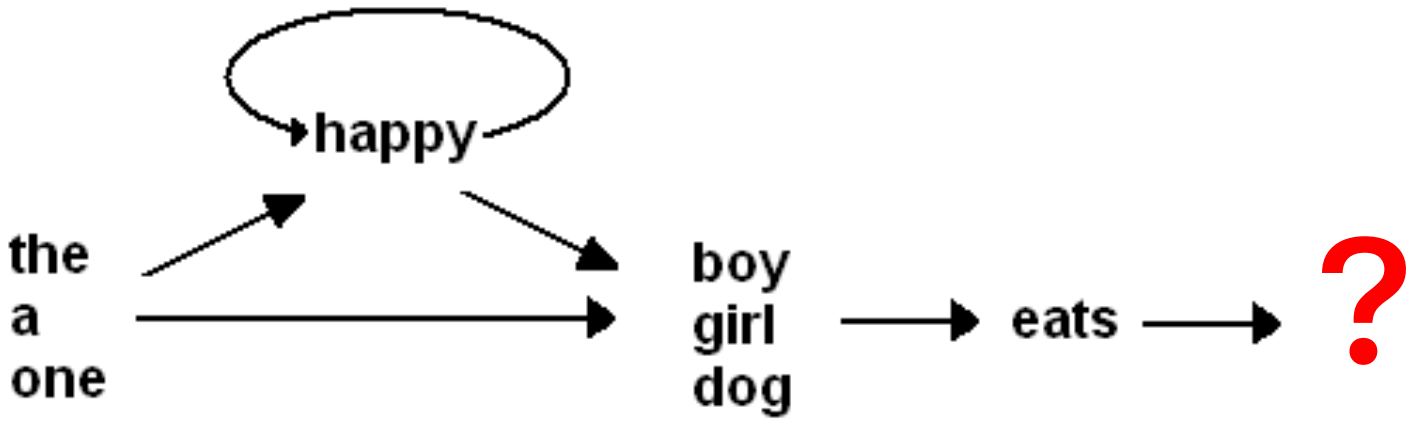


Rules			P	Rules			P
S	→	VP	.05	S	→	VP	.05
VP	→	Verb NP	.20	VP	→	Verb NP NP	.10
NP	→	Det Nominal	.20	NP	→	Det Nominal	.20
Nominal	→	Nominal Noun	.20	NP	→	Nominal	.15
Nominal	→	Noun	.75	Nominal	→	Noun	.75
				Nominal	→	Noun	.75
Verb	→	book	.30	Verb	→	book	.30
Det	→	the	.60	Det	→	the	.60
Noun	→	dinner	.10	Noun	→	dinner	.10
Noun	→	flights	.40	Noun	→	flights	.40

Note: probabilities are calculated PER rule, without taking into account application of other rules for earlier words!

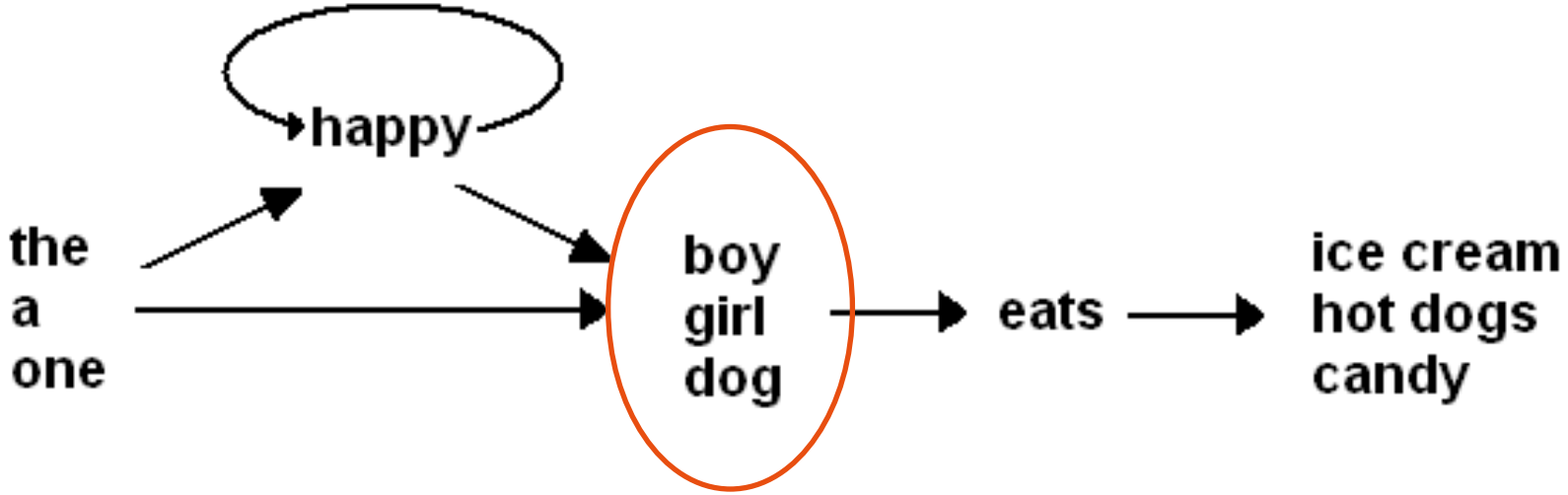

Rule-Based NLP
Statistical NLP
NP (luggage)
VP (passengers)
VP (arrom)
VP (clicked)
next: deep learning

Language as a Sequence Model



Autoregressive modeling –
“Given the previous sequence of words, what comes next?”

Language as a Sequence Model



Not only next-word prediction, but also POS-tagging, Named Entity Recognition (Semantic Role modeling) and other tagging c.q. classifications tasks in NLP, all the way to Abstracting, Machine Translation, Q&A and Chat Bots …

Statistical Models to Choose from

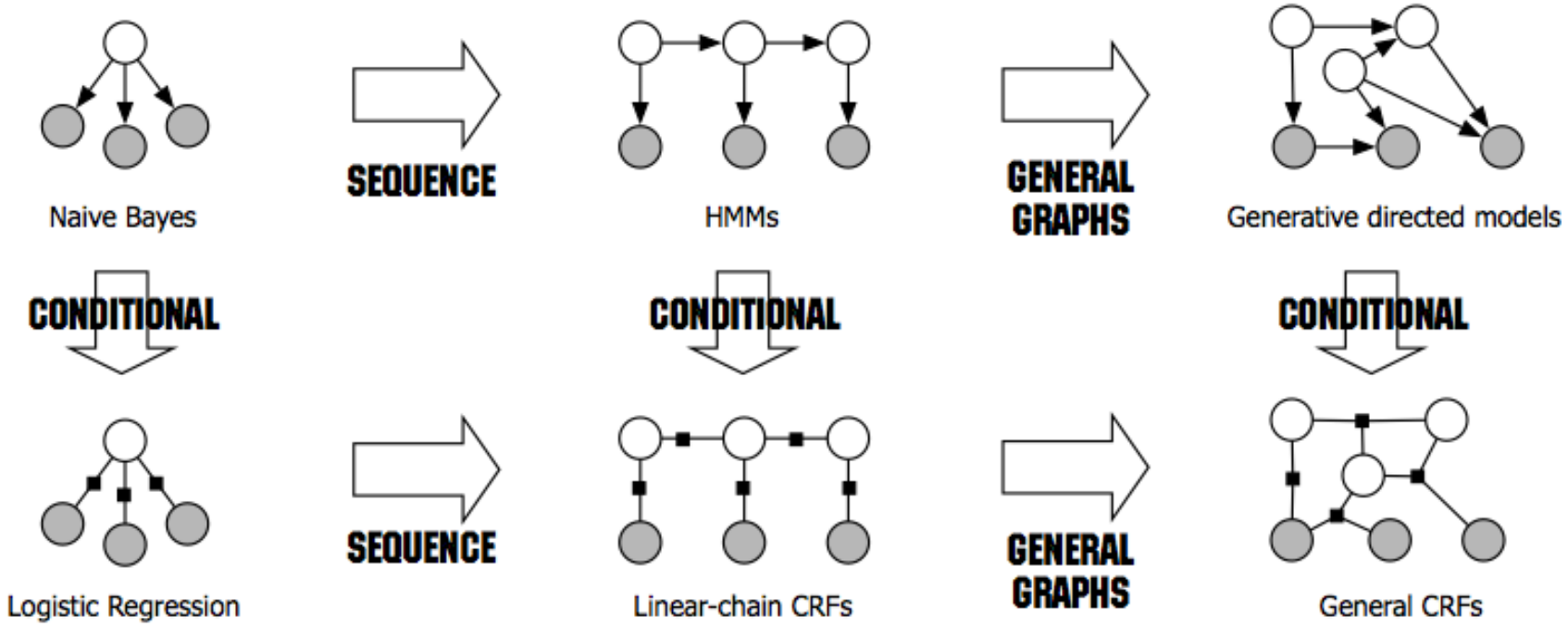


Figure 1.2 Diagram of the relationship between naive Bayes, logistic regression, HMMs, linear-chain CRFs, generative models, and general CRFs.

From: Sutton, C & McCallum, A 2012, 'An Introduction to Conditional Random Fields', Foundations and Trends in Machine Learning, vol. 4, no. 4, pp. 267-373. https://doi.org/10.1561/2200000013

Language is Temporal

- Natural Language is **temporal** by nature.
- Requires **temporal reasoning**.
- Both **left** and **right** context-sensitive.
- **Long distance** (inter-sentential) relations
- So, we require statistics that can deal with sequences and predict correctly what is next over long distances.

So, language as a **"sequence model"**

Teaching note: "No Bayes" here refers to plain classification Bayes, not the sequence-aware variants (HMM, CRF) covered later in this lecture.

Language as a Sequence Problem: would that work?

Writing English using randomly selected characters:

- Between A-Z:

  `QWXVYBJSNKDPLTZURMOAEGCHIXFQWCBVYATRGDEJLHPZOXNMWK`

- With the **frequency of each letter** proportional to its commonness in the **English** language:

  `EER ITHA NTHE RO AET S  ETO NTE ORTA L R MATH ISE THA IOE NR`

- Where the frequency of each letter (and spaces) depends on its commonness in English and the likelihood of a **given letter following the one** that came before it (bi-grams):

  `Thes e ar e wor ds t hat ma y sou nd  qu ite  ran dom  but sti ll  f orm a  k ind o f flo w.`

- With the conditional probability depending on **3 lettters (trigrams or n-grams)** before the random new letter:

  `The quiest of the compained and ther of the matation of you cande the frospeced the when.`

We can also do this with **randomly selected words** based on the **conditional probability of the previous words**:

- **1 word:** `The man walked through the door and found a place to sit by the window.`
- **2 words:** `The man sat quietly by the window, looking out at the trees swaying in the breeze.`
- **3 words:** `The old man sat quietly by the window, watching the sun set behind the distant hills.`

Yes!

So, let's presume language is a sequence problem

- A language model is basically a probability distribution over words or word sequences.
- "There is no grammar. There are only sequences of words."
- Statistical models good for dealing with sequences (a bit more advanced than just n-grams):
  - Naïve Bayes
  - Visual Markov models (VMM),
  - Hidden Markov Models (HMM),
  - Maximum Entropy Markov Models (MEMM), and
  - Conditional Random Fields (CRF).

Teaching note: "No grammar, only sequences" is a deliberately provocative simplification — it's the working assumption for this entire lecture, not a final answer (see slide 91's full picture).

Conditional Probabilities

Naïve Bayes



Maastricht University

Recall a few probability basics

- For events *a* and *b:*
- Bayes' Rule

$$p(a,b) = p(a \cap b) = p(a \mid b)\,p(b) = p(b \mid a)\,p(a)$$

$$p(\bar{a} \mid b)\,p(b) = p(b \mid \bar{a})\,p(\bar{a})$$

$$p(a \mid b) = \frac{p(b \mid a)\,p(a)}{p(b)} = \frac{p(b \mid a)\,p(a)}{\sum_{x=a,\bar{a}} p(b \mid x)\,p(x)}$$

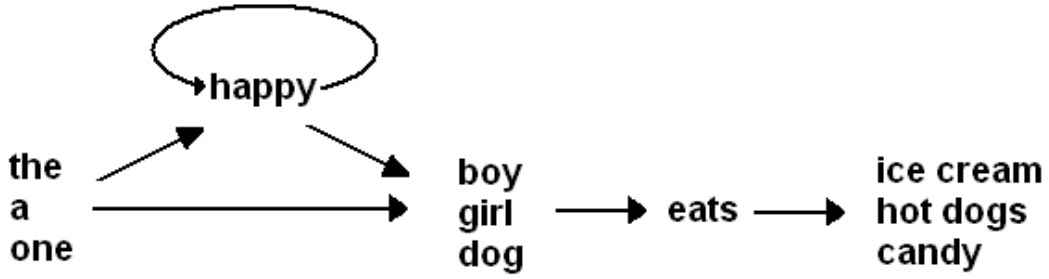
Prior

Posterior

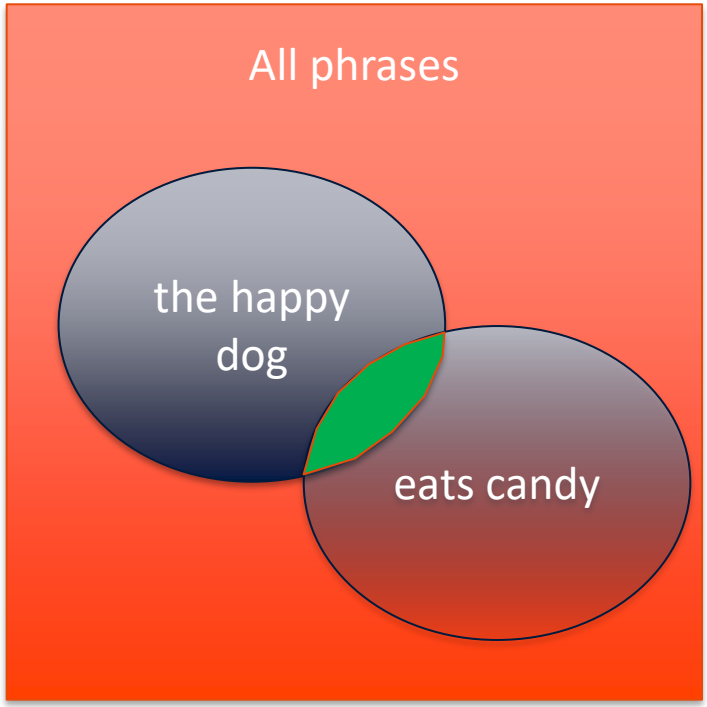
- Odds:

$$O(a) = \frac{p(a)}{p(\bar{a})} = \frac{p(a)}{1 - p(a)}$$

Bayes rule



$S \rightarrow NP\ VP$ or "the happy dog" "eats candy"
Or A:"the happy dog" B:"eats candy"



We want to know $P(B|A)$
We do know $P(A), P(B)$
We know $P(any\ other|B)$
and therefore

$$P(A|B) = 1 - P(any\ other|B)$$

and then,

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

Bayes' Rule for text classification

- For a document *d* and a class *c*

$$P(c,d) = P(c \mid d)P(d) = P(d \mid c)P(c)$$

$$P(c \mid d) = \frac{P(d \mid c)P(c)}{P(d)}$$

Naive Bayes Classifiers

Task: Classify a new instance *d* based on a tuple of attribute values into one of the classes $c_j \in C$

$$d = \langle x_1, x_2, \ldots, x_n \rangle$$

$$c_{MAP} = \underset{c_j \in C}{\operatorname{argmax}} P(c_j \mid x_1, x_2, \ldots, x_n)$$

$$= \underset{c_j \in C}{\operatorname{argmax}} \frac{P(x_1, x_2, \ldots, x_n \mid c_j) P(c_j)}{P(x_1, x_2, \ldots, x_n)}$$

$$= \underset{c_j \in C}{\operatorname{argmax}} P(x_1, x_2, \ldots, x_n \mid c_j) P(c_j)$$

MAP is "maximum a posteriori" = most likely class

Naive Bayes Classifier: Naive Bayes Assumption

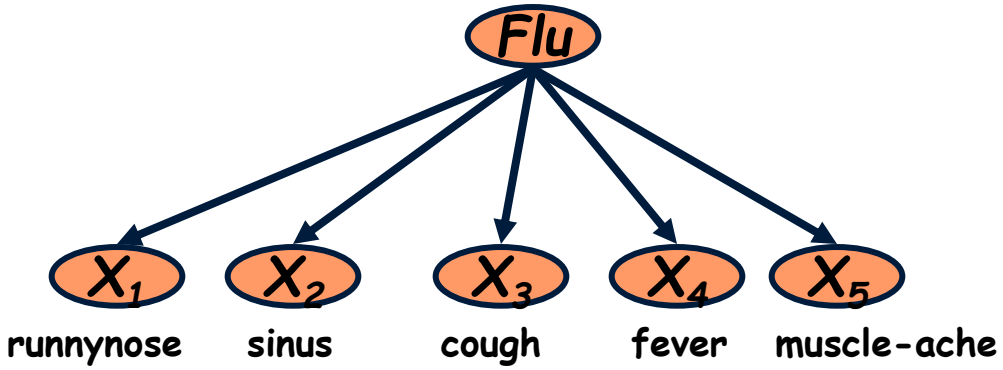
- $P(c_j)$:
  - Can be estimated from the frequency of classes in the training examples.

- $P(x_1,x_2,\ldots,x_n|c_j)$:
  - $O(|X|^n \bullet |C|)$ parameters
  - Could only be estimated if a very, very large number of training examples was available.

Naive Bayes Conditional Independence Assumption:

- Assume that the probability of observing the conjunction of attributes is equal to the product of the individual probabilities $P(x_i|c_j)$.

The Naive Bayes Classifier



- **Conditional Independence Assumption:** features detect term presence and are **independent** of each other **given the class**:

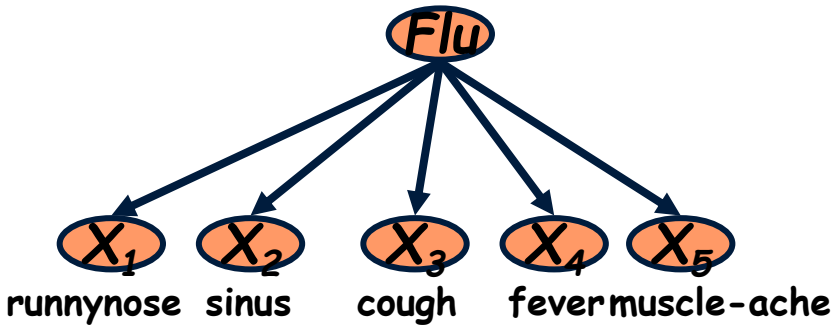
$$P(X_1,\ldots,X_5 \mid C) = P(X_1 \mid C) \bullet P(X_2 \mid C) \bullet \cdots \bullet P(X_5 \mid C)$$

Naïve Bayes & NLP …

- Naïve Bayes presumes independence between features.
- Otherwise, we cannot multiply the probabilities; calculation then become complex.
- If features are word occurrences, can we then state that **NLP and Naïve Bayes work together well? In other words, is the occurrence of words in Natural Language independent?**

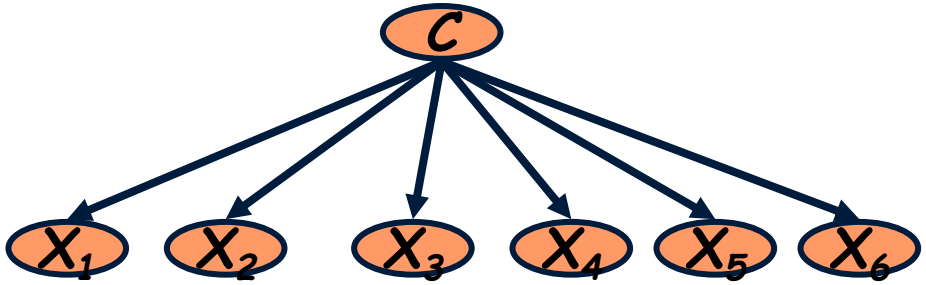
Conditional Independence Assumption: features detect term presence and are independent of each other given the class:

$$P(X_1,\ldots,X_5 \mid C) = P(X_1 \mid C) \bullet P(X_2 \mid C) \bullet \cdots \bullet P(X_5 \mid C)$$



Teaching note: Answer (see speaker notes on this slide): no — word occurrences are topic-dependent, so the independence assumption is technically false, yet Naive Bayes still works well in practice for bag-of-words tasks.

Training the Model



- First attempt: maximum likelihood estimates
  - simply use the frequencies in the data

$$\hat{P}(c_j) = \frac{N(C = c_j)}{N}$$

$$\hat{P}(x_i \mid c_j) = \frac{N(X_i = x_i, C = c_j)}{N(C = c_j)}$$

The Problem with token-level Naïve Bayes (NB)

- Strong independence assumption:
  - Words in a sentence are not independent: their order and local context matter.
  - E.g., when talking about “bank”, depending on the context, words such as *river, water*, *flow*, *fish,* etc will occur versus words such as *loan, Finance, interest,* etc. So, there is no true independence.

- No notion of sequence:
  - NB treats each token as isolated, can’t use “history”.

- Sparse context:
  - If a word never occurred with a label in training, NB has no way to generalize from neighbors.
  - If the word occurs only once, it will get a probability of 1 (label bias).

As a result, NB works for bag-of-words tasks (spam, topic classification) but not for structured problems like tagging, segmentation, or speech recognition.

Teaching note: This is exactly why sequence models (HMM, CRF) exist — NB has no mechanism at all for “what came before,” only for “what's present.”

Sequences: from Unigram to Higher-Order Models

P (● ○ ● ●)

= P (●) P (○ | ●) P (● |● ○)P (● | ● ○ ●)

Unigram Language Models

P (●) P (○) P (●) P (●)

Easy. Effective!

Bigram (generally, *n*-gram) Language Models

P (●) P (○ | ●) P (●| ○) P (●| ●)

Would this then work to model the sequential nature of language?

The Problem with n-gram (tokens) Naïve Bayes

- Still need to assume conditional independence of the labels
  - Overlapping features violate NB's independence assumption → probability estimates become distorted.

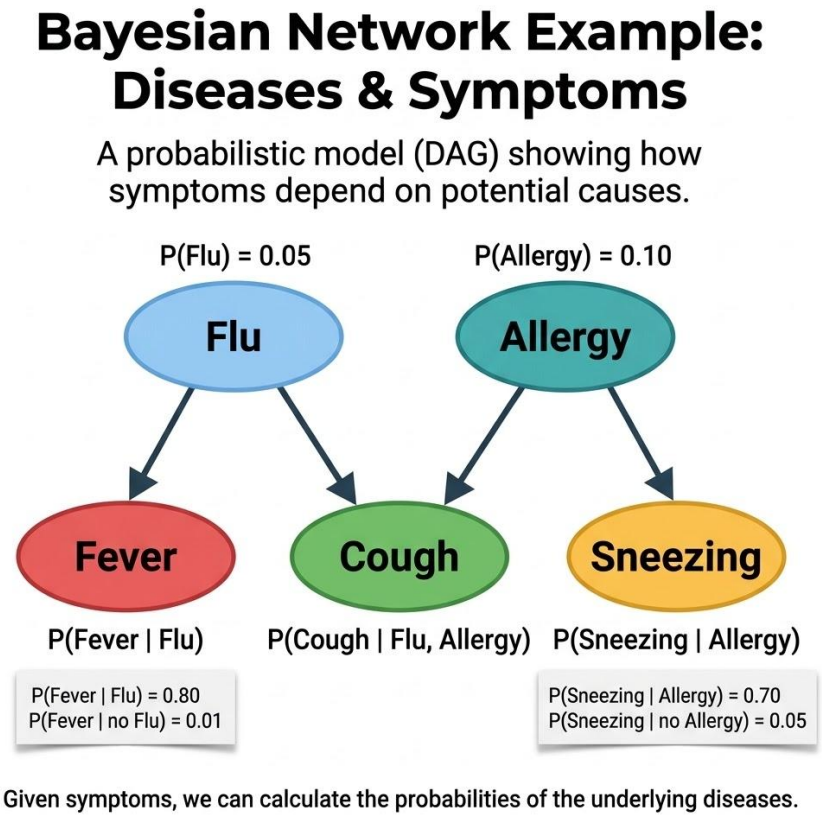
- New complications that are introduced:
  - Data sparsity:
    - number n-grams = $|V|^n$, with V Vocabulary size and n from n-gram.
    - There is never enough training data to cover all possibilities.

  - Not sequence-aware in the right way
    - Only backwards and limited context. Language also depends on what will come later.
    - Memory & computation cost: number of P(*n-gram*|*class*) is huge.
    - N-grams allow for correlation between word, but not between labels:
      - POS example: "Time flies like an arrow".
      - NB with trigrams can look at surrounding words for **flies**, but cannot reason that **flies's** tag should agree with its role given.

  - Fragile generalization:
    - If a new sentence contains an unseen trigram, NB falls back to smoothed prior, often too flat.

What we want is a Bayesian Network

- A Bayesian network (also known as a Bayes network, Bayes net, belief network, or decision network) is a probabilistic graphical model that represents a set of variables and their conditional dependencies via a directed acyclic graph (DAG).
- Bayesian networks are ideal for taking an event that occurred and predicting the likelihood that any one of several possible known causes was the contributing factor.
- For example, a Bayesian network could represent the probabilistic relationships between diseases and symptoms. Given symptoms, the network can be used to compute the probabilities of the presence of various diseases.
- In NLP, Bayesian networks can combine any set of variables to model arbitrary rich conditional dependencies: morphology, syntactic parsing, complex semantic relations, …



Given symptoms, we can calculate the probabilities of the underlying diseases.

See also: https://en.wikipedia.org/wiki/Bayesian_network

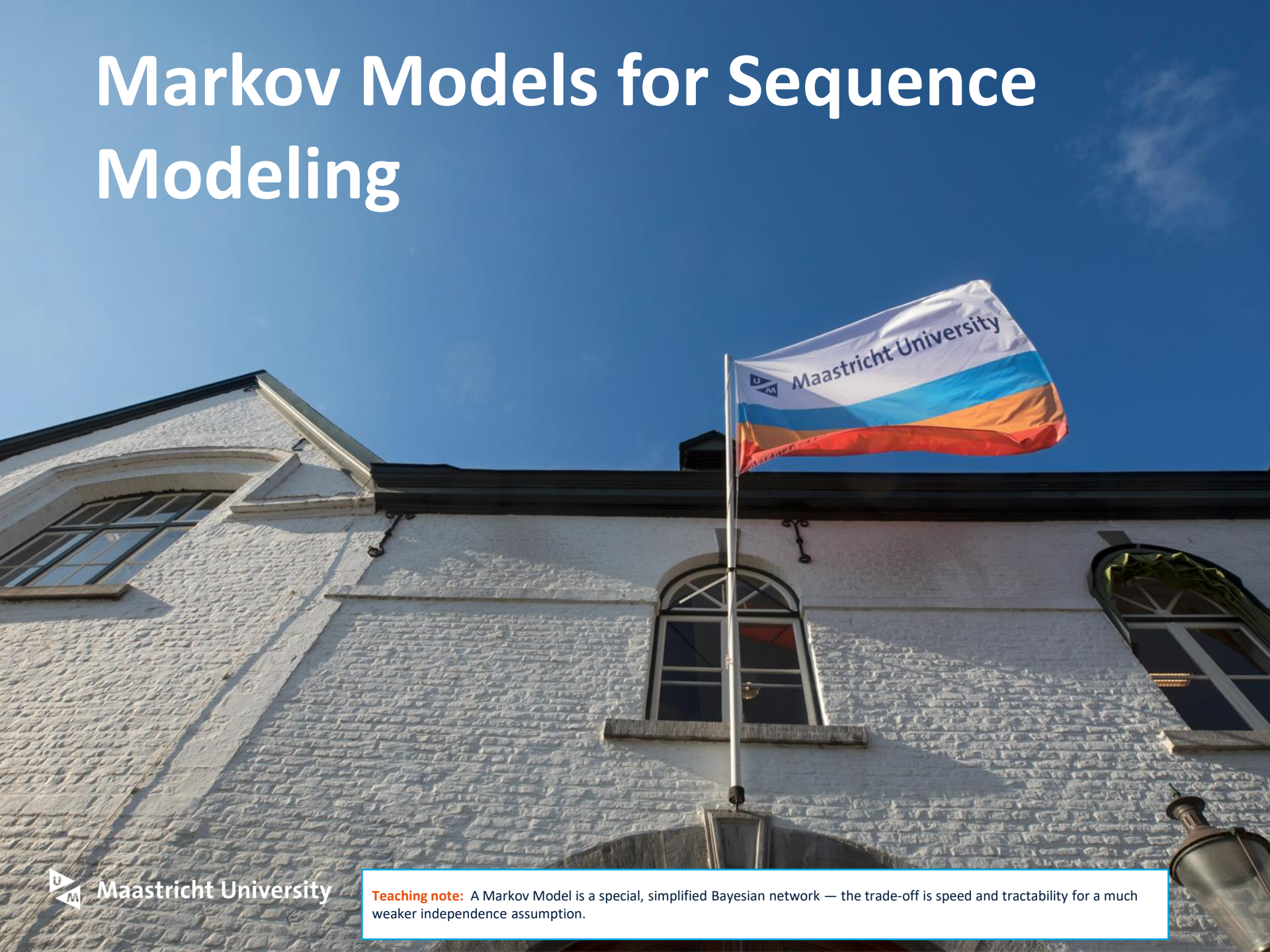
Bayesian Networks

- Bayesian Networks are NP-Hard (like Traveling Salesman).
- Useful when you need to capture many interrelated features (morphology, syntax, metadata) and can afford heavier computation.
- Overkill for basic operations such as POS-tagging, NER or Speech Recognition.

- So, let's take a look at Markov Models, a special (simplified) case of Bayesian Networks.

Teaching note: NP-Hard = provably no efficient exact algorithm exists for general inference — Markov Models (next slides) buy tractability by throwing away most of the graph structure.

Markov Models for Sequence Modeling



Maastricht University

Teaching note: A Markov Model is a special, simplified Bayesian network — the trade-off is speed and tractability for a much weaker independence assumption.

The Markov Condition in General

- The Markov condition, sometimes called the Markov assumption, is an assumption made in Bayesian probability theory, that every node in a Bayesian network is conditionally independent of its non-descendants, given its parents.

- Stated loosely, it is assumed that a node has no bearing on nodes which do not descend from it.

- In a first-order Markov assumption the probability of an observation at time n only depends on the observation at time n-1. A second-order Markov assumption would have the observation at time n depend on n - 1 and n - 2. Etc.



Andrey Andreyevich Markov. 1856-1922

Markov assumption

$$P(S_{i+1}|S_i, S_{i-1}, \dots) = P(S_{i+1}|S_i)$$

The next word only depends on the previous word (which depends on the one before that (which depends on the one before that …

An *autoregressive* formulation

A future event (in this case, the next word) can be predicted using a relatively short history (for the example, one or two words) is called a Markov assumption.

We can look back 7 words, however such a model would require enormous calculations, so we often focus on single words, bi-grams (2 words) or tri-grams (3 words).

The Markov Condition and NLP

What do you think:

"The Markov condition, sometimes called the Markov assumption, is an assumption made in Bayesian probability theory, that every node in a Bayesian network is conditionally independent of its non-descendants, given its parents."

Does the Markov property hold for NLP?

Teaching note: Answer (see speaker notes): no, strictly speaking — word choice depends on future words and other sentences too — but the rest of this lecture tries it anyway because it's tractable.

Markov Chain: "First-order observable Markov Model"

A set of states

$Q = q_1, q_2 \ldots q_N$; the state at time t is q_t

Transition probabilities: (Noun, Verb, Adjective …)

a set of probabilities $A = a_{01}a_{02}\ldots a_{n1}\ldots a_{nn}$.

Each a_{ij} represents the probability of transitioning from state i to state j

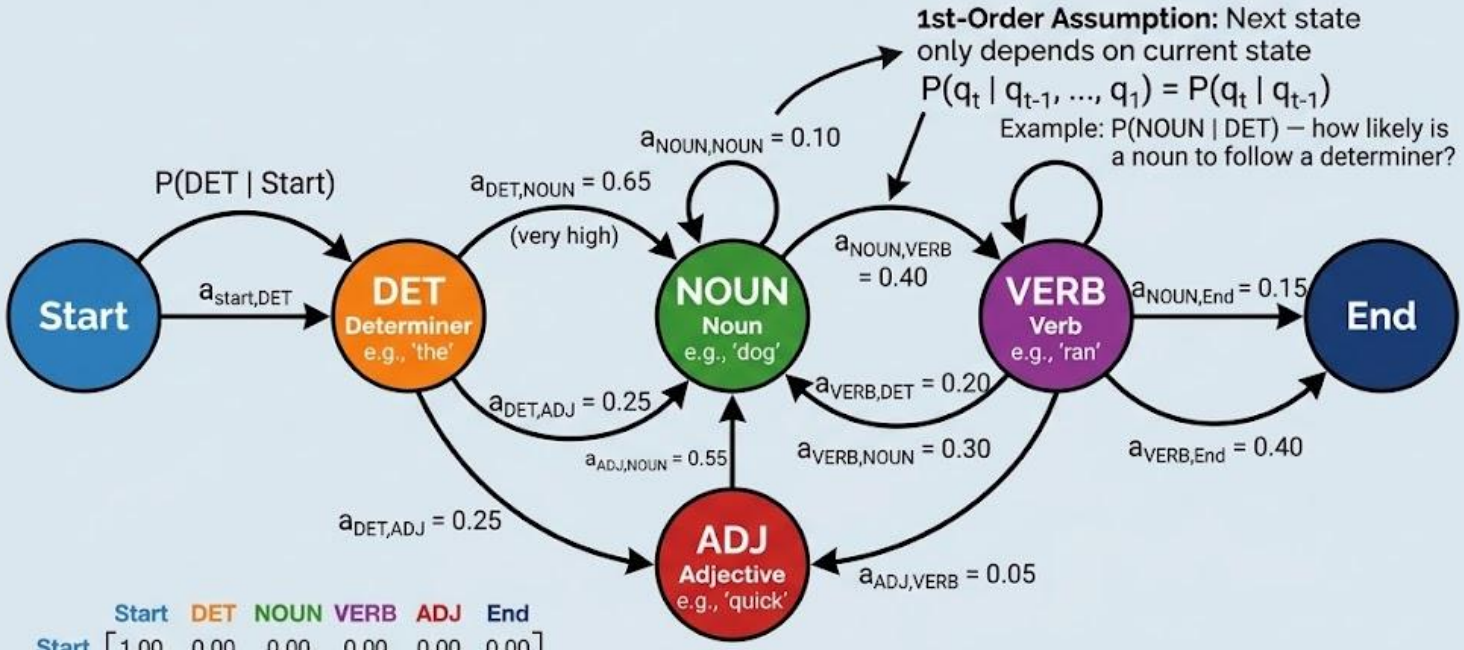
The set of these is the transition probability matrix A

Current state only depends on previous state

$$P(q_i \mid q_1 \ldots q_{i-1}) = P(q_i \mid q_{i-1})$$

Markov Chain: First-Order Observable Natural Language Processing (NLP

State transitions in POS (Part-of-Speech) Tagging



	Start	DET	NOUN	VERB	ADJ	End
Start	1.00	0.00	0.00	0.00	0.00	0.00
DET	0.00	0.00	0.65	0.25	0.25	0.00
NOUN	0.65	0.65	0.50	0.40	0.15	0.15
VERB	0.20	0.20	0.30	0.30	0.05	0.40
ADJ	0.00	0.25	0.25	0.05	0.00	0.00
End	0.00	0.00	0.00	0.00	0.00	1.00

1st-Order Assumption: Next state only depends on current state

$$\rightarrow P(q_t \mid q_{t-1}, \ldots, q_1) = P(q_t \mid q_{t-1})$$

Example: P(NOUN |DET) — how likely is a noun to follow a determiner?

A state is now a POS tag. But how about the words?

From Visual to Hidden Markov Models



Maastricht University

Teaching note: The key upgrade: instead of directly observing states (POS tags), we only observe words and must infer the hidden tag sequence — that's the whole point of the next slides.

VMM versus HMM

- In a VMM we can calculate the probability to move from one state to another.
- A state is for instance NOUN, DETERMINER, ADJECTIVE, VERB, …
- But we are never 100% sure that our previous state decision was correct… and we do not incorporate the information we know: the observations (e.g. the words in the text).

Teaching note: This is the crucial upgrade: a VMM's "state" is directly visible; an HMM's state (the POS tag) must be inferred purely from what IS visible — the words.

Hidden Markov Models (HMM)

- Markov Models are also called Visual Markov Models (VMM) to differentiate them from Hidden Markov Model (HMM).
- HMM are practical tools for speech recognition and text processing. **Also, very popular in gaming, robotics and genomics.**
- Start of usage was in the late 1960s. In the 1980s increased popularity to solve problems that can be defined with a finite number of states and a probability vector of observations for each state.

VMM and HMM analogy:

VMM: Try to predict sequences of head-tail when someone is tossing a coin together with you.

HMM: Same problem, but now that someone is in another room and you cannot see the coin. He or she tells you it is head or tail and you do not know if he or she is lying.
Exact states are thus not clear, so hidden states...

An Example of a HMM Process

Consider two friends, Alice and Bob, who live far apart from each other and who talk together daily over the telephone about what they did that day. Bob is only interested in three activities: walking in the park, shopping, and cleaning his apartment. The choice of what to do is determined exclusively by the weather on a given day. Alice has no definite information about the weather where Bob lives, but she knows general trends. Based on what Bob tells her he did each day, Alice tries to guess what the weather must have been like.

Alice believes that the weather operates as a discrete Markov chain. There are two states, "Rainy" and "Sunny", but she cannot observe them directly, that is, they are *hidden* from her. On each day, there is a certain chance that Bob will perform one of the following activities, depending on the weather: "walk", "shop", or "clean". Since Bob tells Alice about his activities, those are the *observations*. The entire system is that of a hidden Markov model (HMM).
Alice knows the general weather trends in the area, and what Bob likes to do on average. In other words, the parameters of the HMM are known.

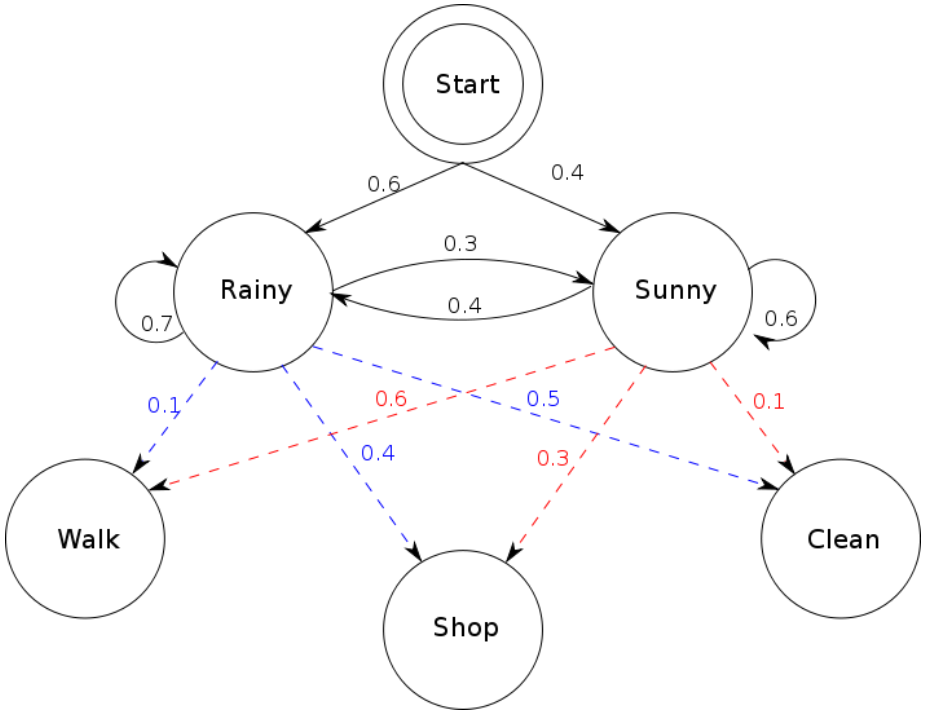


Example Revisited

- States = ('Rainy', 'Sunny') = Y_1, Y_2
- Observations = ('Walk', 'Shop', 'Clean') = O_1,O_2, O_3
- Start-Probabilities = {'Rainy': 0.6, 'Sunny': 0.4} = Π
- Transition Probabilities = {
  - 'Rain' : {'Rainy': 0.7, 'Sunny': 0.3},
  - 'Sun' : {'Rainy': 0.4, 'Sunny': 0.6},

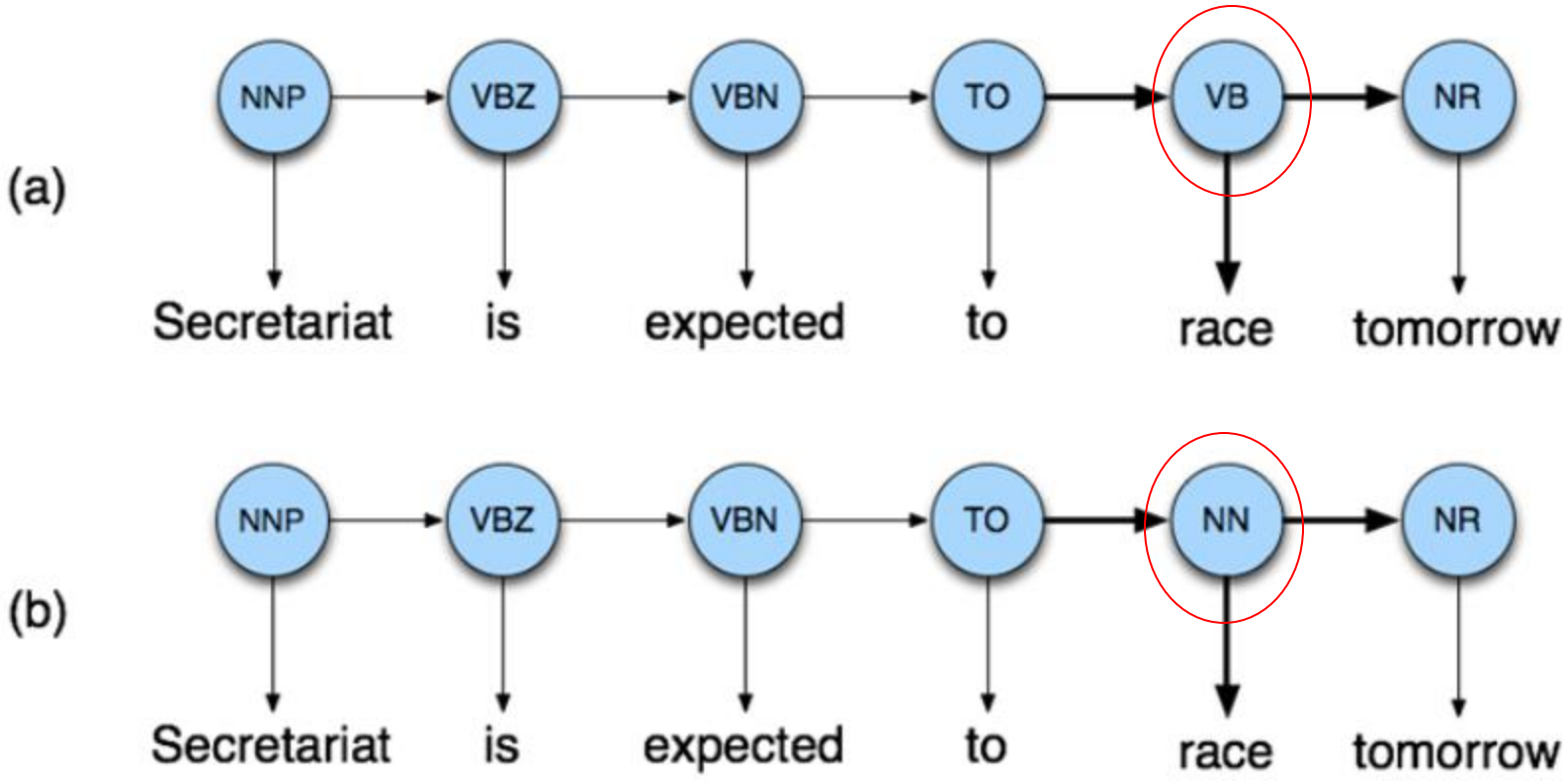
  } = A
- Emission Probabilities = {
  - 'Rainy' : {'Walk': 0.1, 'Shop': 0.4, 'Clean': 0.5},
  - 'Sunny' : {'Walk': 0.6, 'Shop': 0.3, 'Clean': 0.1},

  } = B





Disambiguating "race"



In this context, is RACE a Verb (VB) or a Nominal Noun (NN)?

How can we use this in NLP and Text Mining?

Observations = Words in Natural Language Sentences
States = POS type or entity type or semantic meaning

Use probabilities from HMM for linguistic disambiguation

HMM POS Taggers

The three (3) problems for an HMM translated to POS tagging:

1. Find the probability of an observation sequence = How likely is a test sequence generated by a particular HMM in POS.

2. Find the most likely *state* sequence that we can associate with a particular *observation* sequence = Find the most likely *POS tag* sequence that we can associate with a particular *word* sequence

3. Construct the HMM. Estimate the parameters of the HMM (state transition probability, observation matrix probability, initial state probabilities) were estimated from training data.

Teaching note: These 3 problems map directly to what follows: problem 1→the forward algorithm, problem 2→Viterbi, problem 3→counting from a tagged corpus (slides 40–43).

HMM Definitions

For an HMM we have:

- A set of hidden **states** Y_t (POS tags).
- A set of **observations** O_t (words).

Parameters:

- Initial probabilities:

$$\pi_i = P(Y_1 = i)$$

- Transition probabilities:

$$a_{ij} = P(Y_t = j \mid Y_{t-1} = i)$$

- Emission probabilities:

$$b_j(o) = P(O_t = o \mid Y_t = j)$$

If we *knew* the tag sequence $Y_1, \ldots, Y_n$, the joint probability of tags and words is:

$$P(Y_{1:n}, O_{1:n}) = \pi_{Y_1}\, b_{Y_1}(O_1) \prod_{t=2}^{n} a_{Y_{t-1},Y_t}\, b_{Y_t}(O_t)$$

Usually we don't know the tags; we want:

$$P(O_1, O_2, \ldots, O_n) = \sum_{Y_1,\ldots,Y_n} \pi_{Y_1} b_{Y_1}(O_1) \prod_{t=2}^{n} a_{Y_{t-1},Y_t} b_{Y_t}(O_t)$$

i.e., **sum over all possible tag sequences**.

That's the quantity computed by the **Forward algorithm**.

We also call this the "Observation Probability"

HMM Usage in NLP and Text Mining

1. **Observation Probability**: Find how likely the observation sequence O = O_1, O_2 and O_3 is given our HMM model = select most probable next word in language or determine of sequence of words is valid.

$$P(O_1, \ldots, O_n) = \sum_{Y_1, \ldots, Y_n} \pi_{Y_1} b_{Y_1}(O_1) \prod_{t=2}^{n} a_{Y_{t-1}, Y_t} b_{Y_t}(O_t)$$

2. **State Sequence**: Uncover the most likely state sequence (X,Y, …) that could have generated the observation sequence O = O_1, O_2 and O_3 = select most probable POS tag, entity name or Semantic tagging given a certain sentence

Viterbi algorithm, see next slides

3. **Parameter Estimation** for HMM λ = {**A**, **B**, **Π**}.

Counting various occurrences of O and Y in a corpus, see next slides.

HMM POS Taggers Problem 3: training aka parameter estimation

- Sample training data:

  The/AT bear/NN is/VB on/IN the/AT move/NN

AT = article, VB = Verb, NN = singular noun, IN = preposition, etc.

These can be obtained from tagged corpora such as the Brown and Penn Treebank.

HMM Taggers : Training tag transitions: A

- HMM Parameters are two matrices and one vector: λ = {**A**, **B**, **Π**}

1. Matrix **A** represents the state transition probability matrix. Element a_{ij} is the probability of transitioning from state (tag) *i* to state (tag) *j*. $C(t_i,t_j)$ and $C(t_i)$ are the counts of the number of times a tag *j* follows the tag *i* and the number of occurrences of tag *i*.
   If i, j, and k represent the determiner, noun and verb tags respectively, then a_{ij} will be greater than a_{ik} in most training sets.
   In matrix A, number of rows = number of columns.

$$a_{ij} = \frac{C(t_i, t_j)}{C(t_i)} = p(t_j \mid t_i)$$

HMM Taggers: Training Observations for a tag: B:

2. The second matrix **B** is the probability of an observation (word) given a state (tag). An element b_{ij} contains the probability of emitting word *j* for state (tag) *i*. $C(w_j,t_i)$ is the number of times word *j* was associated with tag *i*. Then the conditional probability is:

$$b_{ij} = \frac{C(w_j, t_i)}{C(t_i)} = p(w_j \mid t_i)$$

HMM-Taggers: Training Initial State: Π

3. The probability vector **Π** is the probability of starting in a particular state (tag). $C(s_i)$ is the number of times a sentence begin with state (tag) *i* and N is the number of sentences in the training set, then the initial probability is:

$$\pi_i = \frac{C(S_i)}{N}$$

HMM POS Taggers Problem 3: Viterbi Algorithm: Find the most likely *POS tag* (Y_1-Y_n) sequence that we can associate with a particular *word* sequence (O_1-O_n)

- Given an HMM and an observation sequence $O_{1:n} = O_1, O_2, \ldots, O_n$, find the **most likely sequence of hidden states** $Y_{1:n}$(e.g., POS tags) that could have produced it. So:

We want:

$$\hat{Y}_{1:n} = \arg\max_{Y_{1:n}} P(Y_{1:n} \mid O_{1:n})$$

Since:

$$P(Y_{1:n} \mid O_{1:n}) \propto P(Y_{1:n}, O_{1:n})$$

it's equivalent to maximising the joint probability:

$$\pi_{Y_1} b_{Y_1}(O_1) \prod_{t=2}^{n} a_{Y_{t-1},Y_t} b_{Y_t}(O_t)$$

The Viterbi Algorithm (e.g. recursion)

Define:

$$\delta_t(j) = \max_{Y_{1:t-1}} P(Y_{1:t-1}, Y_t = j, O_{1:t})$$

= the probability of the **best path** ending in state j at time t.

Initialization:

$$\delta_1(j) = \pi_j \, b_j(O_1)$$

$$\psi_1(j) = 0$$

Recursion (for $t = 2..n$):

$$\delta_t(j) = b_j(O_t) \cdot \max_i \left[\delta_{t-1}(i) \, a_{ij}\right]$$

Store the argmax:

$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) a_{ij}\right]$$

Termination:

$$P^* = \max_j \delta_n(j)$$

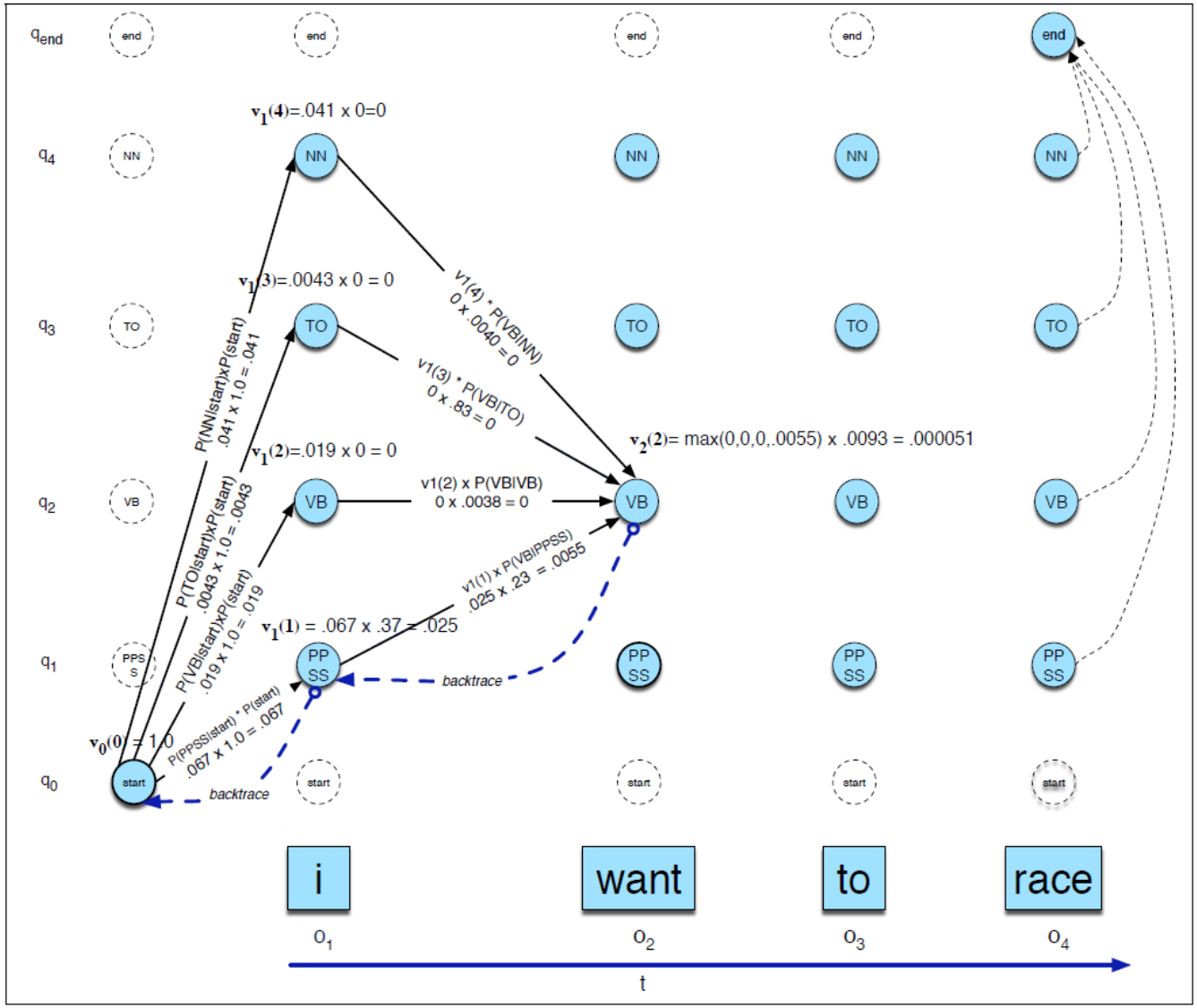
$$\hat{Y}_n = \arg\max_j \delta_n(j)$$

Backtrace:

$$\hat{Y}_t = \psi_{t+1}(\hat{Y}_{t+1}), \quad t = n-1, n-2, \ldots, 1$$

Intuition: At each position, we keep only the best partial path to each state. Because of the Markov property, this is optimal: future probability depends only on the current state, not on how we arrived there.

A Graphical View of the Viterbi Algorithm



Maastricht University

“Secretariat is expected to race tomorrow.”

- Race = Verb or Nominal Noun?

States (tags) we care about:

- `NN` – Noun
- `VB` – Verb
- (others like `DT`, `JJ`, … are present but we'll simplify)

Observations (words):

$$O_1 = \text{Secretariat},\ O_2 = \text{is},\ O_3 = \text{expected},\ O_4 = \text{to},\ O_5 = \text{race},\ O_6 = \text{tomorrow}$$

Model ingredients:

- Initial probs $\pi(s) = P(Y_1 = s)$
- Transition probs $a_{ij} = P(Y_t = j | Y_{t-1} = i)$
- Emission probs $b_j(o) = P(O_t = o | Y_t = j)$

```
time t →   1          2          3          4          5          6
words →  Secretariat  is       expected   to         race       tomorrow
tags
NN         [π·b(NN|Sec)]
VB         [π·b(VB|Sec)]
             ↘             ↘ ...
```

At **position 5 ("race")** there are two main branches:

```
css

... → (to / TO)
          ↘
        race/NN
        race/VB
```

Each path has a score:

$$\delta_t(j) = b_j(O_t) \max_i [\delta_{t-1}(i) a_{ij}]$$

Why "race" → Verb has higher score

Emission evidence

- $b_{\text{VB}}(\text{race})$ = P(word="race"|tag=VB)
- $b_{\text{NN}}(\text{race})$ = P(word="race"|tag=NN)

Both may be reasonable (since *race* is ambiguous).

Transition evidence

Look at the tag before "race": `TO` ("to").

- P(VB | TO) is **very high** (infinitival "to" usually followed by a verb).
- P(NN | TO) is very low (after "to" rarely a bare noun).

So:

$$\delta_5(VB) = b_{VB}(\text{race}) \cdot \delta_4(\text{TO}) \cdot a_{\text{TO},VB}$$

$$\delta_5(NN) = b_{NN}(\text{race}) \cdot \delta_4(\text{TO}) \cdot a_{\text{TO},NN}$$

Because $a_{\text{TO},VB} \gg a_{\text{TO},NN}$, the VB branch dominates.

"Probability lattice" (simplified numbers)

t	word	tag	prev-score × trans × emit	δ
4	to	TO	–	0.42
5	race	VB	0.42 × 0.8 × 0.05	**0.0168**
5	race	NN	0.42 × 0.05 × 0.06	0.00126

> The verb path's score is >10× the noun path.

At $t = 5$ the algorithm stores:

$$\psi_5(VB) = \text{TO}, \quad \psi_5(NN) = \text{TO}$$

During backtracking from the highest score at $t = 6$, the path chosen is:

```
scss

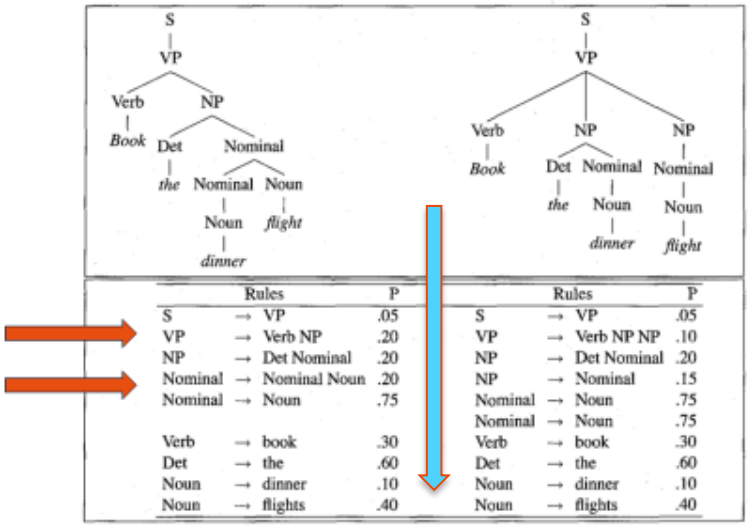
NN (Secretariat)
VBZ (is)
VBN (expected)
TO (to)
VB (race)   ← chosen
NN (tomorrow)
```

The transition probability from “to” to Verb is the decisive factor. Even if “race” is also a noun, the syntactic context (to …) makes the Verb path much more probable, so the Viterbi algorithm outputs: Y_5=VB

```
Secretariat  is     expected     to        race        tomorrow
   NN        VBZ      VBN       TO    { VB | NN }        NN
                                    ↗
                          Viterbi picks VB because:
                              - P(race|VB) is reasonable
                              - P(VB | TO) >> P(NN | TO)
```

Remember the PCFG approach … this works much better!

Disambiguating with statistics: "book a flight that serves dinner" versus "book a flight on behave of "the dinner""



Rules			P
S	→	VP	.05
VP	→	Verb NP	.20
NP	→	Det Nominal	.20
Nominal	→	Nominal Noun	.20
Nominal	→	Noun	.75
Verb	→	book	.30
Det	→	the	.60
Noun	→	dinner	.10
Noun	→	flights	.40

Rules			P
S	→	VP	.05
VP	→	Verb NP NP	.10
NP	→	Det Nominal	.20
NP	→	Nominal	.15
Nominal	→	Noun	.75
Nominal	→	Noun	.75
Verb	→	book	.30
Det	→	the	.60
Noun	→	dinner	.10
Noun	→	flights	.40

In PCFG the differences between the probabilities for different meaning are small. In the HMM approach, the differences are much larges, so the model, is more certain.

Maastricht University

Note: probabilities are calculated PER r
consideration application of other rules

Token	First Tag	Second Tag	Third Tag
Plastic	0.914 : nn	0.086 : jj	0.000 : np
signs	0.945 : nns	0.055 : vbz	0.000 : nn$
work	0.632 : vb	0.367 : nn	0.001 : jj
around	0.929 : in	0.071 : rb	0.000 : vb
the	1.000 : at	0.000 : jj	0.000 : vb
clock	0.996 : nn	0.004 : rb	0.000 : jj
.	1.000 : .	0.000 : np	0.000 : nn

Maastricht University

Viterbi with Log Probability

- Since the raw probability scores are usually very small in real examples (with many states).
- Therefore, we introduce a new concept: the joint log probability. The joint probability of the sequence of words found in the sentence will rank the probability the set of tags for a sentence.
- By taking the log of the summations instead of the raw multiplications, we avoid the probabilities to become smaller than what floating points in a computer can handle (floating point underflow). Additions are also computationally more efficient than multiplications of floating points.

Log Probability	Tag Sequence
-80.460	Plastic_nn signs_nns work_vb around_in the_at clock_nn ._.
-81.346	Plastic_nn signs_nns work_nn around_in the_at clock_nn ._.
-83.624	Plastic_nn signs_nns work_vb around_rb the_at clock_nn ._.

Viterbi with Log Probability

Define:

$$\delta_t(j) = \max_{Y_{1:t-1}} P(Y_{1:t-1}, Y_t = j, O_{1:t})$$

= the probability of the **best path** ending in state j at time t.

Initialization:

$$\delta_1(j) = \pi_j \, b_j(O_1)$$

$$\psi_1(j) = 0$$

Recursion (for $t = 2..n$):

$$\delta_t(j) = b_j(O_t) \cdot \max_i \left[\delta_{t-1}(i)\, a_{ij}\right]$$

Store the argmax:

$$\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) a_{ij}\right]$$

Termination:

$$P^* = \max_j \delta_n(j)$$

$$\hat{Y}_n = \arg\max_j \delta_n(j)$$

Backtrace:

$$\hat{Y}_t = \psi_{t+1}(\hat{Y}_{t+1}), \quad t = n-1, n-2, \ldots, 1$$

Let

- $\pi_j = P(Y_1 = j)$ (start),
- $a_{ij} = P(Y_t = j \mid Y_{t-1} = i)$ (transition),
- $b_j(o) = P(O_t = o \mid Y_t = j)$ (emission),
- $O_{1:n}$ the observations,
- $\delta_t(j)$ the best path score up to time t ending in state j.

Define log-parameters:

$$\ell\pi_j = \log \pi_j, \quad \ell a_{ij} = \log a_{ij}, \quad \ell b_j(o) = \log b_j(o).$$

Initialization

$$\boxed{\delta_1(j) = \ell\pi_j + \ell b_j(O_1)}, \qquad \psi_1(j) = \text{nil}.$$

Recursion (for $t = 2, \ldots, n$)

$$\boxed{\delta_t(j) = \ell b_j(O_t) \; + \; \max_i \left[\delta_{t-1}(i) + \ell a_{ij}\right]},$$

$$\boxed{\psi_t(j) = \arg\max_i \left[\delta_{t-1}(i) + \ell a_{ij}\right]}.$$

Termination

$$\boxed{\log P^* = \max_j \delta_n(j)}, \qquad \boxed{\hat{Y}_n = \arg\max_j \delta_n(j)}.$$

Backtrace

$$\hat{Y}_t = \psi_{t+1}(\hat{Y}_{t+1}), \quad t = n-1, \ldots, 1.$$

Break



Limitations of HMM POS Taggers

The use of HMM to model natural language has off course it's limitations:

- HMM's do not account for relationships with words that are not *immediate neighbors*.
  - **Long distance relationships** (which is one word in a first order HMM) do occur in natural language and cannot be modeled with a first-order HMM.
  - The probability of the tag of a word *x* is based on the **previous words**' tag as well as the tags of words from a text window centered on *x (FSG equivalent)*.

- **HMM are still generative models**, so you need (plenty) data for every possible occurrence of combinations of words and POS tags, otherwise, it does not store it in the model (probability = 0) or if it occurs only once: bias (p = 1 for all such cases).

Teaching note: "Only looks one word back" is precisely the limitation MEMM and CRF (slides 61 onward) are built to remove.

How can we get more context sensitivity in our models?

- Higher order Markov Models.
- Rules based tagging with HMM. Rules for long term relations.
- Maximum Entropy Markov Models – long term relations in feature functions.
- Conditional Random Fields (CRF) – long term relations and other linguistic phenomena implemented using feature functions.

Extending the HMM to trigrams

$$\text{Trigrams} \quad \hat{P}(t_i|t_{i-1},t_{i-2}) = \frac{C(t_{i-2},t_{i-1},t_i)}{C(t_{i-2},t_{i-1})}$$

$$\text{Bigrams} \quad \hat{P}(t_i|t_{i-1}) = \frac{C(t_{i-1},t_i)}{C(t_{i-1})}$$

$$\text{Unigrams} \quad \hat{P}(t_i) = \frac{C(t_i)}{N}$$

So far, these were all first order Markov models (one word forward). One can also look at m^{th} order Markov models which examine m words before or after a word.

A higher order HMM will be much more **complex to calculate**. It also requires enormous amounts of training data to cover each possible combination between POS-tags and words (combinatorial explosion).

Rule Based Tagging and HMM: A quick fix

- A rules-based tagger can solve some of the limitations of a HMM by making a few assumptions:
  - Many words in English have just one POS (see next slide).
  - A small fraction of words have more than two parts of speech.
  - Content words (noun, adjectives, verbs, and adverbs) make up to 99% of all dictionary words (see next slide).
  - Since content words are also a high percentage of all the words, most mis-tagged words belong to one of these four parts of speech.
  - A word wrongly tagged may be more than one part of speech.
  - Words that belong to a single tag in the dictionary are always tagged correct.
  - Words not in the dictionary may be mis-tagged.
  - Some combinations of POS are more common than others (see 2 slides further).

Rule based Tagging: look at vocabulary

No. of Tags	Percentage of Words
1	94.97
2	4.63
3	0.34
4	0.05
5	0.01

Part of Speech	Percentage of Words
Noun	75.1
Adjective	14.0
Verb	7.4
Adverb	3.0
Others	0.5

Rule Based Tagging: real-world word frequencies

- Despite statement previous slide: although single tags may be non-ambiguous, combinations are almost always ambiguous : percentage of all words in a document that have more than one POS for a particular combination.

- This is 97% of all the word combinations can have more than one tag.

- a=adjective, n=noun, v=verb, r=adverb.
- nv = most common: *labor, quarrel, cheer, raise*. E.g. *labor*: noun is more likely.
- an = *second, senior*, *orange*, *tutorial*. E.g. Adjective is the more likely.
- Few words such as *rough*, *quiet*, and *home* have 4 possible tags: highly ambiguous!

Tag Combination	Word Percentage	Tag Combination	Word Percentage
nv	48.57	av	2.19
an	34.77	anr	1.35
anv	4.57	anrv	0.72
ar	4.52	nr	0.72

Rule Based Tagging Decision Tree

- Traverse tree for each word.
- Single tags for words in dictionary are easiest.
- Words with multiple tags: use HMM to find most common tag based on a rule.
- If we cannot use a rule, then just use most likely tag.
- If we do not know word: check if it is a name (Capital?), then probably noun.
- Else: check based on suffix what the highest probable tag would be.

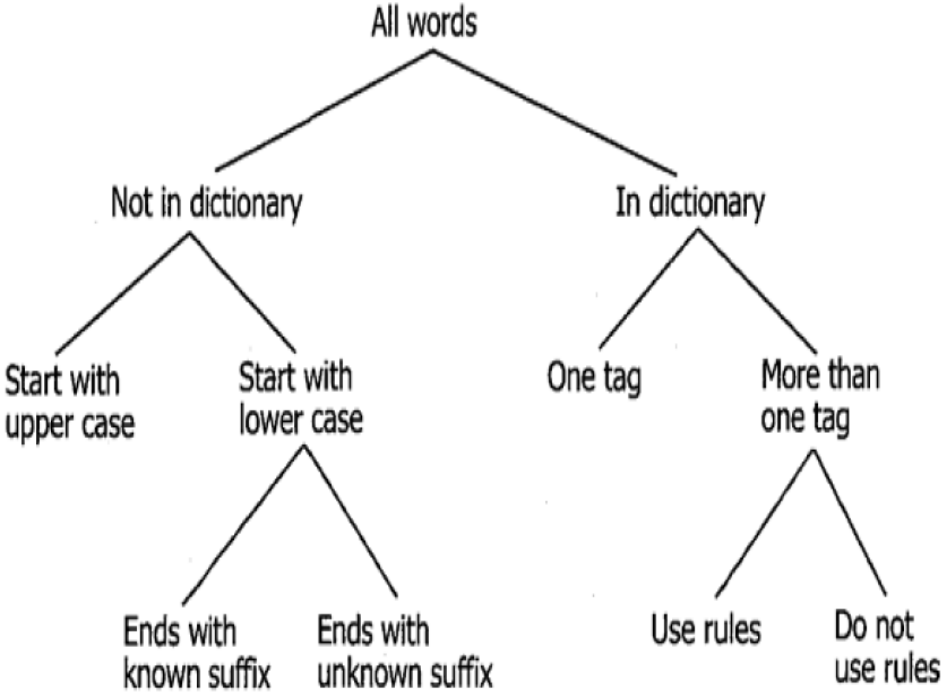


FIGURE 4.9 Decision tree to assign POS tags.

Teaching note: This rule-based patchwork is a stopgap — MEMM/CRF (next section) replace hand rules with learned feature weights.

Rules-based tagging requires lots of manual efforts and maintenance. Also, sensitive to changes in language and to errors in language usage.

Maximum Entropy Markov Modeling (MEMM) and Conditional Random Fields (CRF)



Maastricht University

Teaching note: MEMM and CRF trade the HMM's rigid word-only lookup tables for arbitrary feature functions — this is what lets them use capitalization, suffixes, and long-range context.

MEMM & CRF: Move away from words only …

- Instead of working with words, and entropy over words, we will define feature functions.

- This can be anything you can imagine: structural features, short & long term left or right relations, punctuation, POS roles, semantic role or even just being a “WORD”.

Examples of Basic Linguistic Features

Feature	*About feature*	*Regular Expression*
Initcaps	First letter is in uppercase	[A-Z]\w+
Initcapsalpha	First letter is in uppercase. Second letter is in lowercase.	[A-Z][a-z]\w+
All caps	All letters are in uppercase	[A-Z]+
Caps mix	Mixture of uppercase and lowercase letters	[A-Za-z]+
Has digit	Protein name has a number in the middle	\w+[0-9]\w+
Single digit	Ranges from 0-9	[0-9]
Double digit	Two digit numbers	[0-9][0-9]
Natural numbers	Any natural number	[0-9]+
Real numbers	Decimal numbers /numbers with comma	[-0-9]+[.,]+[0-9.,]+
Has dash	Protein name with dash in middle	\w+ - \w+
Init dash	First character is a dash	- \w+
End dash	Last character is a dash	\w+ -
Alphanumeric (starts with alphabet)	Combination of alphabets and numbers. First character is an alphabet.	\w+ [A-Za-z] \w+ [0-9] \w+
Alphanumeric (starts with number)	Combination of alphabets and numbers. First character is a number.	\w+ [0-9] \w+ [A-Za-z] \w+
Roman letter	Any roman letter	[IVXDLCM]+
Has Roman	Any roman letter in the middle	\w+ [IVXDLCM]+ \w+
Greek	Any Greek letter	\w+ [aßYÖE]
Has Greek	Any Greek letter in middle	\w+ [aßYÖE] \w+
Punctuation	Protein name with punctuation	

Note: \w+ - any number of characters.

More Advanced Linguistic Features

Lexicon Features	Grammar Features	
t		Binary-specific features
$b(t)$	ρ	
$\langle t, w \rangle$	$\langle b(p(r_p)), ds(w_s) \rangle$	$\langle b(p(r_p)), ds(w_{s-1}, dsw_s) \rangle$
$\langle t, lc(w) \rangle$	$\langle b(p(r_p)), ds(w_e) \rangle$	**PP feature:**
$\langle b(t), w \rangle$	**unary?**	if right child is a PP then $\langle r, w_s \rangle$
$\langle b(t), lc(w) \rangle$	**simplified rule:**	**VP features:**
$\langle t, ds(w) \rangle$	base labels of states	if some child is a verb tag, then rule,
$\langle t, ds(w_{-1}) \rangle$	**dist sim bigrams:**	with that child replaced by the word
$\langle t, ds(w_{+1}) \rangle$	all dist. sim. bigrams below	
$\langle b(t), ds(w) \rangle$	rule, and base parent state	Unaries which span one word:
$\langle b(t), ds(w_{-1}) \rangle$	**dist sim bigrams:**	
$\langle b(t), ds(w_{+1}) \rangle$	same as above, but trigrams	$\langle r, w \rangle$
$\langle p(t), w \rangle$	**heavy feature:**	$\langle r, ds(w) \rangle$
$\langle t, unk(w) \rangle$	whether the constituent is "big"	$\langle b(p(r)), w \rangle$
$\langle b(t), unk(w) \rangle$	as described in (Johnson, 2001)	$\langle b(p(r)), ds(w) \rangle$

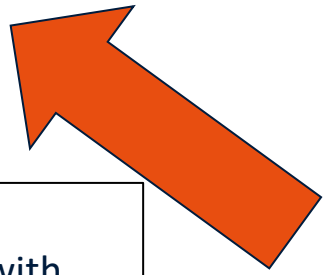
Table 1: Lexicon and grammar features. w is the word and t the tag. r represents a particular rule along with span/split information; ρ is the rule itself, r_p is the parent of the rule; w_b, w_s, and w_e are the first, first after the split (for binary rules) and last word that a rule spans in a particular context. All states, including the POS tags, are annotated with parent information; $b(s)$ represents the base label for a state s and $p(s)$ represents the parent annotation on state s. $ds(w)$ represents the distributional similarity cluster, and $lc(w)$ the lower cased version of the word, and $unk(w)$ the unknown word class.

Source: https://nlp.stanford.edu/jrfinkel/papers/acl08_discpcfg.pdf

MEMM and CRF Feature Functions

- Define a **constraint function** *f*: X x X -> R.
- Common practice to make *f*(x,y) a binary-values trigger function with *C*->y_i
- *C* is for instance: "Mr. leads not to a sentence end."
- *C* can be **anything you can come up with!**

$$f(x,y) = \begin{cases} 1, if\ C(x) \quad and\ y = y_i \\ 0, otherwise \end{cases}$$



Help us to deal with long-term right and left context dependencies! No longer only looking back (left) as with HMM

Teaching note: This is the single biggest conceptual jump in the lecture: from "what word is this" (HMM) to "any property of the entire input I can define" (feature functions).

The Intuition behind MEMM & CRF

HMMs are **generative**: they model the *joint* distribution

$$P(x_{1:n}, y_{1:n}) = P(y_1) \prod_{t=2}^{n} P(y_t \mid y_{t-1}) P(x_t \mid y_t)$$

Limitations:

1. **Emission tables** $P(x_t|y_t)$ are huge (one parameter per word–tag pair).
   → Hard to exploit rich signals (capitalisation, suffixes, word shape, previous words...).
2. They assume:
   - Observation at t depends only on tag y_t.
   - Transition depends only on y_{t-1}.
3. They don't directly optimise tagging accuracy; they optimise likelihood of joint data.

We'd like to:

- Condition directly on the observation sequence.
- Use **arbitrary features** of context (prefixes, previous words, is-capitalised, etc.).

Maximum entropy means: among all models that fit the feature expectations from our data, choose the one that makes no other assumptions — the most uniform consistent model. It's not about adding noise; it's about avoiding unjustified certainty."

> Keep the **Markovian** state structure,
> but instead of a fixed emission table, predict each tag y_t with a **maximum entropy / logistic model** using any features of the context and previous tag.

So:

$$P(y_t \mid y_{t-1}, x)$$

is no longer a table — it's an exponential model over **features**.

Why Maximum Entropy?

What "Maximum Entropy" really means in statistics

- In information theory, **entropy** of a distribution p is:

$$H(p) = -\sum_{x} p(x) \log p(x)$$

- The **Maximum Entropy Principle** (Jaynes, 1957) says:

> Among all distributions that satisfy certain constraints, choose the one with the **largest entropy**.

Why?

Because it is the *least biased* distribution consistent with the facts you know — you add **no extra structure** beyond your constraints.

So "maximum entropy" ≠ "randomness for its own sake."

It means: *be as uniform as possible while honouring the evidence.*

Suppose you know:

- Some expected values of **feature functions** (e.g., counts of "word ends in *ing* and tag=VBG" in training data).

Among all conditional distributions $P(y|x)$ that match those expected counts, the MaxEnt solution is:

$$P(y|x) = \frac{\exp(\sum_k \theta_k f_k(x, y))}{Z(x)}$$

That is exactly a **logistic / exponential** model.

> We don't assume any other correlations — we just use features we have, and otherwise stay as uniform as possible.

Intuition behind Maximum Entropy

- Think of each feature as a *constraint* ("this pair should have probability X").
- There are infinitely many distributions consistent with those constraints.
- The MaxEnt principle picks the one that **spreads leftover probability mass as evenly as possible**.

So we get:

- Certainty **only where data/constraints support it**.
- Otherwise, keep probabilities flat.

> Maximum entropy does **not** mean we inject noise.
> It means we avoid *inventing* structure not supported by evidence.

In fact, MaxEnt models are:

- The **most cautious** model given what we know.
- Equivalent to saying: "Assume independence beyond what features capture."

Imagine you're filling in probabilities on a table:

y	x	???
You fix a few cells because features tell you their expected values.		
For the rest, you leave them as even as possible → **no unwarranted peaks or holes.**		

So, MEMM can be defined as

Where does this come from? Next slides …

- MEMMs use a MaxEnt (log-linear) classifier for each step:

$$P(y_t \mid y_{t-1}, x) = \frac{\exp(\theta \cdot f(x, y_{t-1}, y_t))}{\sum_{y'} \exp(\theta \cdot f(x, y_{t-1}, y'))}$$

- The "maximum entropy" part is the theoretical justification for using this exponential form:
  - It's the highest-entropy distribution that satisfies the feature expectation constraints learned from data.

Maximum Entropy ≠ "maximal randomness."

It's the most honest probability distribution: perfectly reflects your evidence, and nowhere else pretends to know more than it does.

Maximal Entropy Markov Modeling

Calculated from training data

Feature functions 0 (if false) or 1 (if true)

- We can then calculate the probability distribution of a constraint function *f* as p(*f*) and P(*f*) the distribution of all p(*f*)'s:

$$p(f) = \sum_{x,y} p(x,y) f(x,y) = P(f) = P(x,y) f(x,y)$$

- In practice, we cannot calculate the true expectation, so we must use the empirical value calculated by summing over the training examples N:

$$P_E(f) = \frac{\sum_{i=1}^{N} \sum_{y \in Y} p(y \mid x_i) f(x_i, y)}{N} = P_E(f) = \frac{\sum_{i=1}^{N} f(x_i, y_i)}{N}$$

Maximal Entropy Modeling

So: we want to find the model with the **highest entropy** for certain **selected constraint feature** functions, which is the model with $\lambda=\text{argmax}_{\lambda}\,\Psi(\lambda)$ and (Feldman, pp 140):

$$p_\lambda(y \mid x) = \frac{1}{Z_\lambda(x)} \exp\left(\sum_k \lambda_k f_k(x, y)\right),$$
$$\Psi(\lambda) = -\sum_{i=1..N} \log Z_\lambda(x)/N + \sum_k \lambda_k P_E(f_k),$$

where $Z_\lambda(x)$ is a normalizing constant determined by the requirement that $\Sigma_{y \in Y} p_\lambda(y \mid x) = 1$. Finally, we pose the *dual* optimization problem

$$\lambda^* = \text{argmax}_\lambda \Psi(\lambda).$$

Feldman, Ronen & Ronen, & Sanger, & James,. (2007). The text mining handbook: Advanced approaches in analyzing unstructured data.

Example of MEMM for POS Tagging

- Secretariat/NN is/VB expected/VB to/TO race/?? Tomorrow/NN

		f1	f2	f3	f4	f5	f6
VB	f	0	1	0	1	1	0
VB	w		.8		.01	.1	
NN	f	1	0	0	0	0	1
NN	w	.8					-1.3

Figure 6.19 Some sample feature values and weights for tagging the word *race* in Eq. 6.79.

Example feature functions

$$f_1(c,x) = \begin{cases} 1 & \text{if } word_i = \text{“race”} \ \& \ c = \text{NN} \\ 0 & \text{otherwise} \end{cases}$$

Another feature would be whether the previous word has the tag *TO*:

$$f_2(c,x) = \begin{cases} 1 & \text{if } t_{i-1} = \text{TO} \ \& \ c = \text{VB} \\ 0 & \text{otherwise} \end{cases}$$

$$f_3(c,x) = \begin{cases} 1 & \text{if suffix}(word_i) = \text{“ing”} \ \& \ c = \text{VBG} \\ 0 & \text{otherwise} \end{cases}$$

$$f_4(c,x) = \begin{cases} 1 & \text{if is_lower_case}(word_i) \ \& \ c = \text{VB} \\ 0 & \text{otherwise} \end{cases}$$

Example feature functions and MEMM disambiguation

Value FF for training data sample

- First 2 lines are FF and Weights for "TO RACE" were RACE is VB.
- Second 2 lines are FF and Weight for RACE as NN

This is weight derived from training data

		f1	f2	f3	f4	f5	f6
VB	f	0	1	0	1	1	0
VB	w		.8		.01	.1	
NN	f	1	0	0	0	0	1
NN	w	.8					-1.3

Figure 6.19 Some sample feature values and weights for tagging the word *race* in Eq. 6.79.

Example

Use this to fill in:

$$p_\lambda(y \mid x) = \frac{1}{Z_\lambda(x)} \exp\left(\sum_k \lambda_k f_k(x, y)\right),$$
$$\Psi(\lambda) = -\sum_{i=1..N} \log Z_\lambda(x)/N + \sum_k \lambda_k P_E(f_k),$$

where $Z_\lambda(x)$ is a normalizing constant determined by the requirement that $\Sigma_{y \in Y} p_\lambda(y \mid x) = 1$. Finally, we pose the *dual* optimization problem

$$\lambda^* = \text{argmax}_\lambda \Psi(\lambda).$$

- Now select the class with the highest probability for P(c|x) over all classes c from C.

$$P(NN|x) = \frac{e^{.8}e^{-1.3}}{e^{.8}e^{-1.3} + e^{.8}e^{.01}e^{.1}} = .20$$

$$P(VB|x) = \frac{e^{.8}e^{.01}e^{.1}}{e^{.8}e^{-1.3} + e^{.8}e^{.01}e^{.1}} = .80$$

MEMM can condition many features of the input such as punctuation, morphology and capitalization as well.

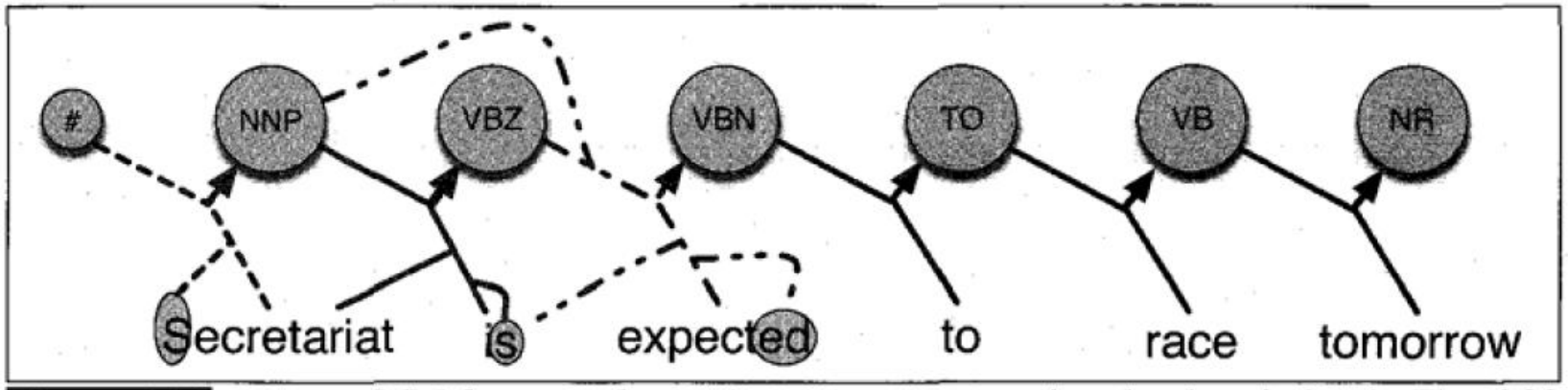
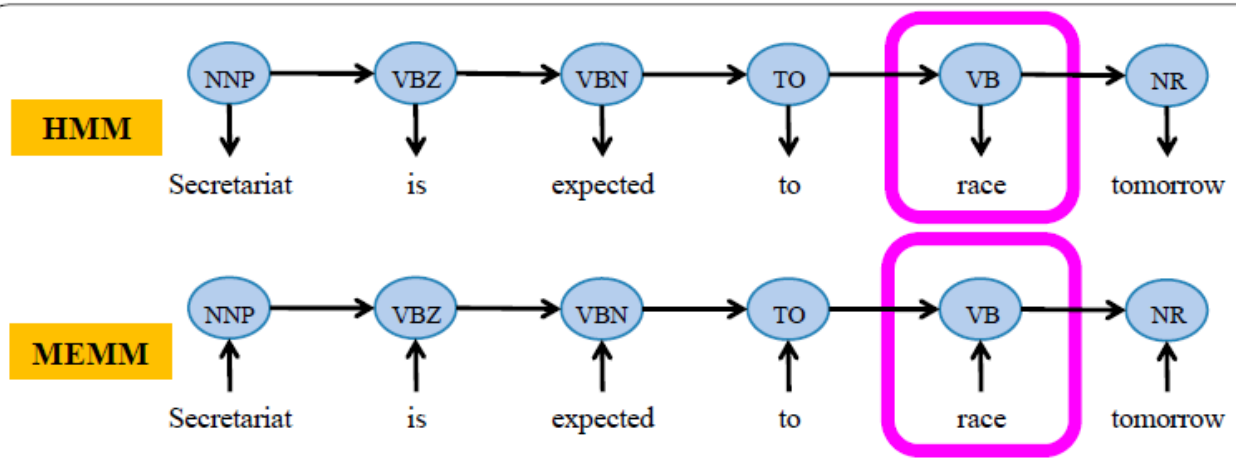


Figure 6.21 An MEMM for part-of-speech tagging, augmenting the description in Fig. 6.20 by showing that an MEMM can condition on many features of the input, such as capitalization, morphology (ending in *-s* or *-ed*), as well as earlier words or tags. We have shown some potential additional features for the first three decisions, using different line styles for each class.



HMM	MEMM
"Generative" models ➔ joint probability **p(words, tags)** ➔"generate" input (in addition to tags) ➔ but we need to predict tags, not words!	"Discriminative" or "Conditional" models ➔ conditional probability **p(tags | words)** ➔"condition" on input ➔ Focusing only on predicting tags
Probability of each slice = emission * transition = p(word_i | tag_i) * p(tag_i | tag_i-1) = ➔ Cannot incorporate long distance features	Probability of each slice = p(tag_i | tag_i-1, word_i) or p(tag_i | tag_i-1, all words) ➔ Can incorporate long distance features

Problems MEMM

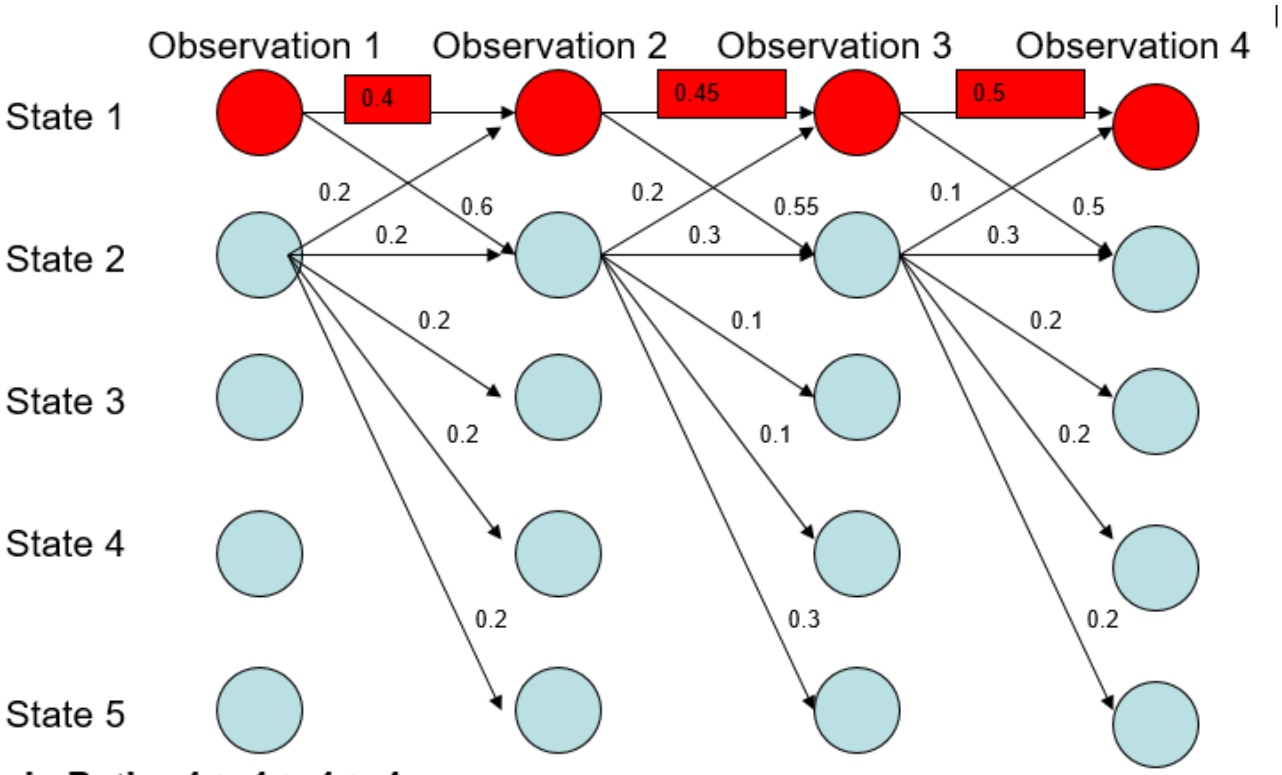
Label bias problem:

- Still needs **lots** of data for **all** possible occurrences in order to build a reliable model (like HMM).
- When there is not enough, certain transition may get probability (close to) 1 and will always prevail despite the input.

$$P_E(f) = \frac{\sum_{i=1}^{N} \sum_{y \in Y} p(y \mid x_i) f(x_i, y)}{N} = P_E(f) = \frac{\sum_{i=1}^{N} f(x_i, y_i)}{N}$$

Teaching note: Watch for this exact problem again on the next 2 slides with a concrete worked example (states with 2 vs. 5 outgoing transitions).

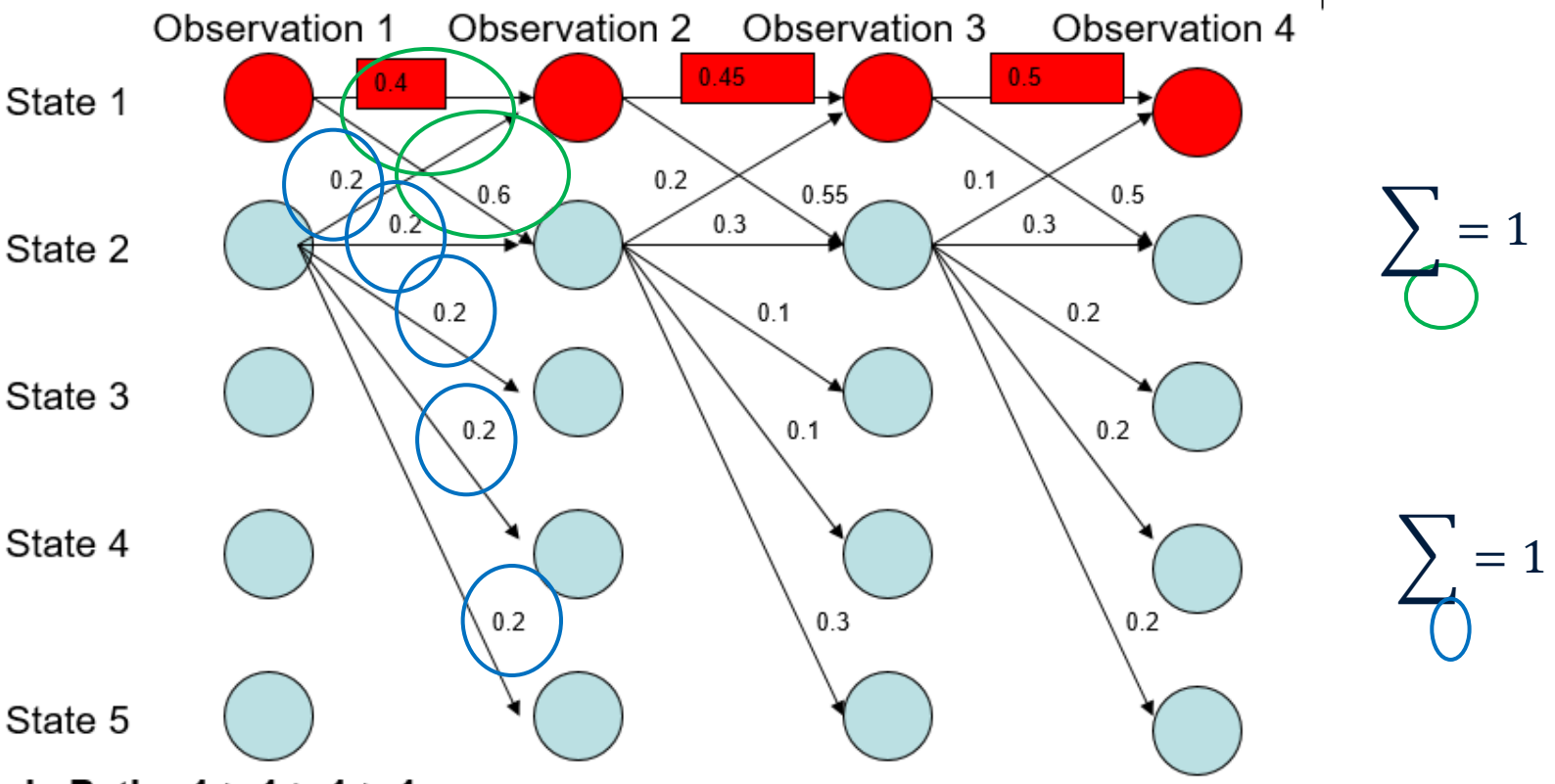
The Label Bias Problem with MEMM



Most Likely Path: 1-> 1-> 1-> 1

- Although locally it seems state 1 wants to go to state 2 and state 2 wants to remain in state 2.

State 1 has only two transitions but state 2 has 5 = label bias problem. Go from local normalization to global.



Most Likely Path: 1-> 1-> 1-> 1

- Although locally it seems state 1 wants to go to state 2 and state 2 wants to remain in state 2.

State 1 has only two transitions but state 2 has 5 = label bias problem. Go from local normalization to global.

Conditional Random Fields (CRF)

- The label bias problem is addressed in Conditional Random Fields, which we will discuss now …

- MEMM is like deciding at each street corner where to go, normalizing among visible exits — but ignoring the full route's difficulty. CRF says: "Let's evaluate all possible routes' total quality, then pick or learn from the best."

- **CRF** is the maximum-entropy model over entire label sequences.
  It fixes the label bias problem by normalizing globally, not locally.
  The price: heavier training. But decoding is as efficient as Viterbi.

- A CRF is like a MEMM but instead of scoring each step independently and normalizing locally, it scores the **entire tag sequence** and normalizes globally.

Teaching note: Same street-corner analogy is the basis of Exercise 2's third question — useful to have ready for the exercises.

How to include the Entire Tag Sequence

- Keep the *log-linear features* idea (like MEMM),
- But evaluate them over the **whole path** $(y_{1:n})$ at once:

The entire path

$$\text{Score}(y_{1:n}, x) = \sum_{t=1}^{n} \theta \cdot f(y_{t-1}, y_t, x, t)$$

Then convert scores to probabilities with a **single partition function**:

$$P(y_{1:n} \mid x) = \frac{\exp(\text{Score}(y_{1:n}, x))}{Z(x)}$$

where:

$$Z(x) = \sum_{y'} \exp(\text{Score}(y', x))$$

> So all candidate sequences compete against each other globally.

CRF – Formal Definition

Let $\bar{x}$ is a sequence of words and $\bar{y}$ a corresponding sequence of n tags:

$$\sum_{y'} \exp(\text{Score}(y', x))$$

Rewritten as ..

$$P(\bar{y} \mid \bar{x}; \bar{w}) = \frac{\exp(\bar{w} \cdot F(\bar{x}, \bar{y}))}{\sum_{\bar{y}' \in Y} \exp(\bar{w} \cdot F(\bar{x}, \bar{y}'))}$$

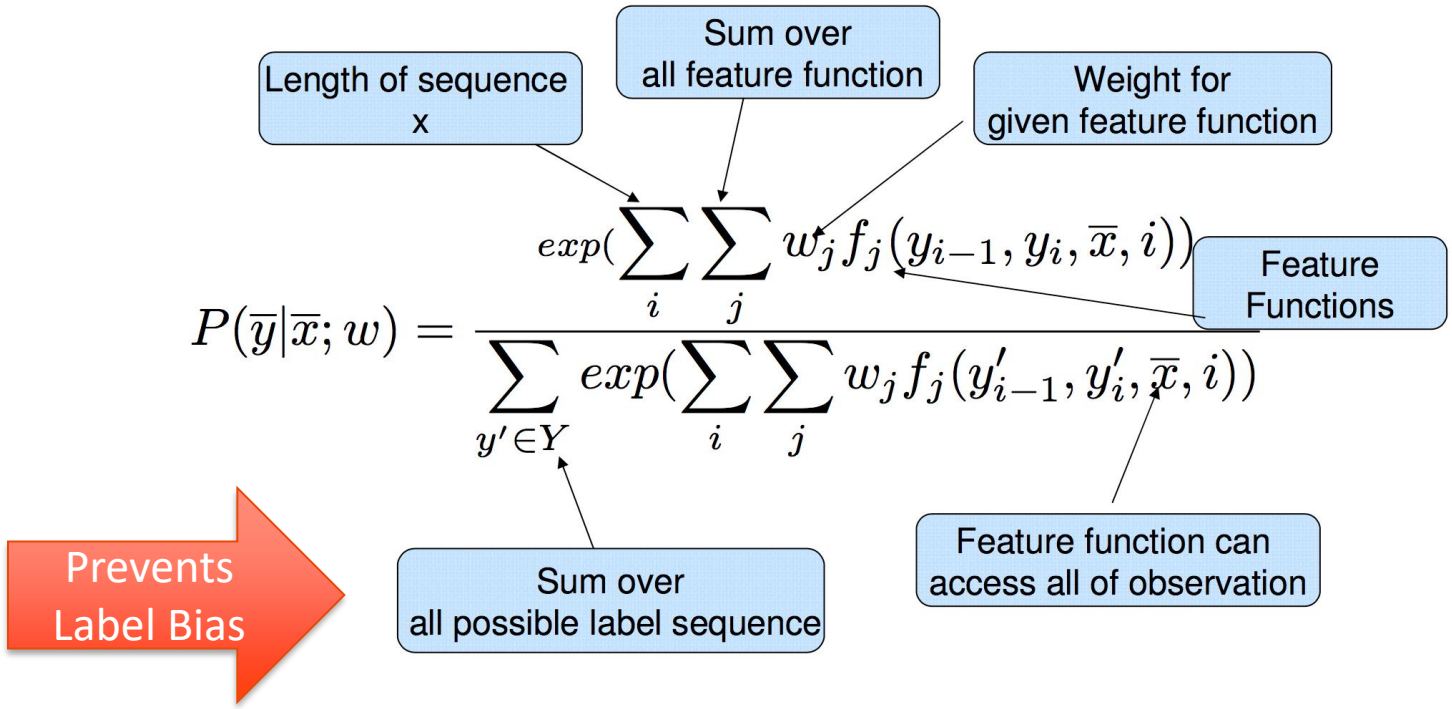
F will represent a global feature vector defined by a set of feature functions $f_1, \ldots, f_d$, where each feature function f_j can analyse the whole $\bar{x}$ sequence, the current y_i and previous y_{i-1} positions in the $\bar{y}$ labels sequence, and the current position i in the sentence:

$$F(\bar{x}, \bar{y}) = \sum_i f(y_{i-1}, y_i, \bar{x}, i)$$

Set *w* by supervised training.

w is weight vector setting importance for each feature function f(**x**,**y**) set with supervised training.

CRF Formal Definition



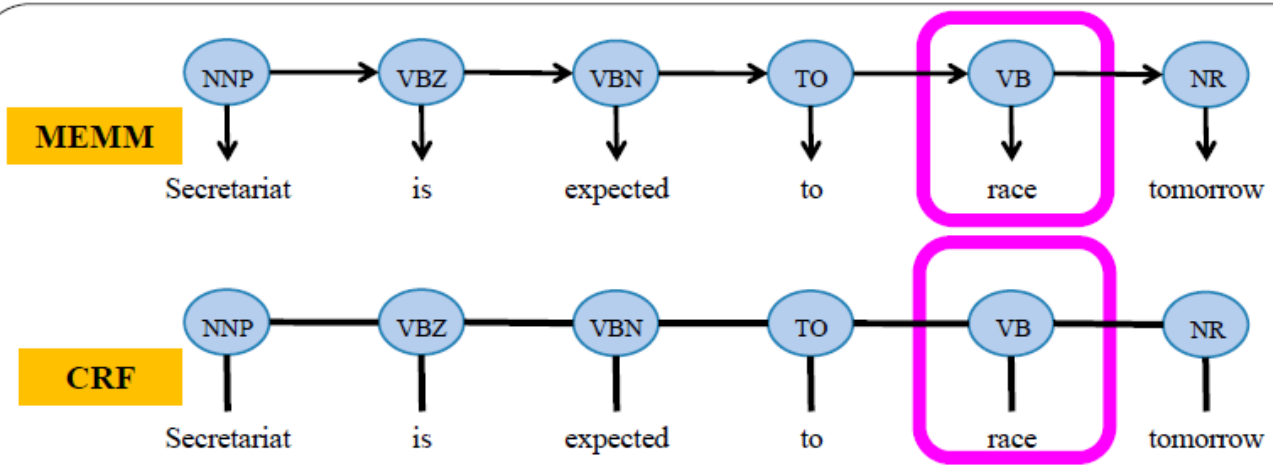
Source: Wikipedia. See also Feldman pp. 142 for more explanations.

But this comes at a cost …

Aspect	MEMM	CRF
Normalisation	local (per state)	global (over all sequences)
Training cost	simple (per-position logistic)	expensive (need forward–backward each gradient step)
Inference	Viterbi (like HMM)	Viterbi (same complexity as HMM)
Parameter estimation	convex but heavy (gradient + DP)	still convex, but slower
Memory	small	must store feature scores per edge

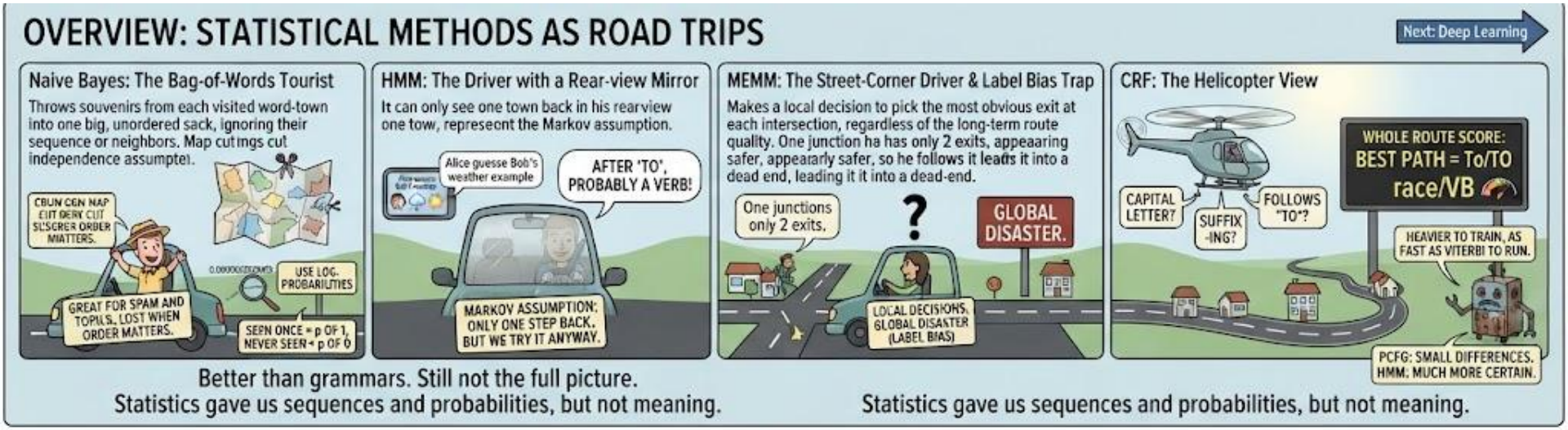
> The big cost is **training**: you must run a forward–backward algorithm on every sentence to compute the global partition function $Z(x)$ and its gradient.

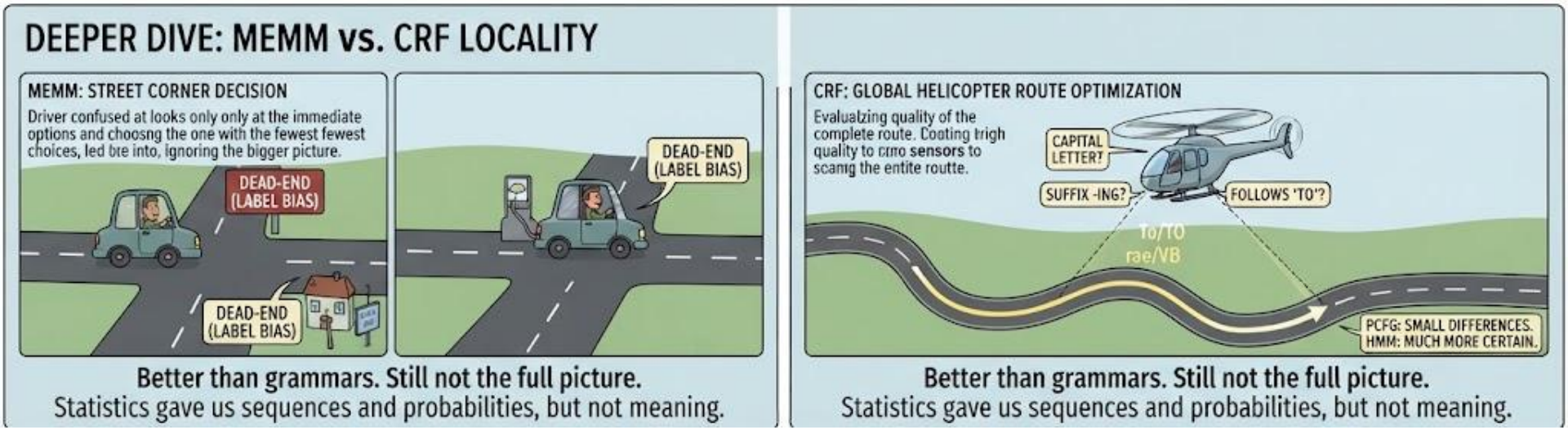
HMM became popular in the mid 1980's. CRF's by the mid 1990. Increase in computational power allowed researchers to move on from HMM to CRF via MEMM.



MEMM	CRF
Directed graphical model	Undirected graphical model
"Discriminative" or "Conditional" models ➔ conditional probability **p(tags | words)**	
Probability is defined for each slice = P (tag_i | tag_i-1, word_i) or p (tag_i | tag_i-1, all words)	Instead of probability, ***potential (energy function)*** is defined for each slide = ϕ (tag_i, tag_i-1) * ϕ (tag_i, word_i) or ϕ(tag_i, tag_i-1, all words) * ϕ (tag_i, all words)
➔ Can incorporate long distance features	

Another way to look at this …





But CRFs Still have Limitations

CRFs are still a powerful structured prediction tool, but compared to modern deep-learning–based models, they have clear limitations:

- **Manual Feature Engineering**: CRFs rely on explicit features $f(y_{t-1}, y_t, x, t)$. You must design and extract these by hand: prefixes, suffixes, word shapes, gazetteers, etc. Deep models (RNNs, CNNs, Transformers) learn features automatically from raw text or embeddings.

- **Limited Representation Power**: CRFs are linear in the features:
  - Score$(x, y) = \theta \cdot F(x, y)$
  - They cannot easily capture:
    - Complex nonlinear interactions between context and labels,
    - Hierarchical or latent structure,
    - Long-distance dependencies (beyond what features encode).
  - Deep models learn rich, hierarchical representations (e.g., via self-attention).

Teaching note: Every limitation named here (manual features, linear scoring, fixed window) is solved by the deep learning models coming in the next lecture.

More CRF Limitations

- **Context Window is Fixed**: Standard CRFs typically look at a small local window of words around position t. To incorporate long-distance context, you need explicit "skip" or "history" features, which quickly explode in number. Transformers capture context across the entire sentence (or even document) automatically.

- **Training & Scalability**: CRF training is $O(n|Y|^2)$ per gradient step (with forward–backward). With large tag sets or very long sequences, this is slow. Deep models may also be heavy, but they benefit from GPU acceleration and scale better with data size.

- **Handling Raw or Continuous Input**: CRFs assume discrete, sparse features. Incorporating embeddings or dense vectors directly is awkward (you must treat them as fixed inputs or pretrain them separately). Neural models can directly ingest embeddings and fine-tune them jointly.

- **Generative Capacity**: CRFs are discriminative: $P(y|x)$. They don't model $P(x)$ or generate text; they only label given sequences. Deep models can be used as generative (e.g., seq2seq, language models) or discriminative.

- **Data Efficiency versus Representation**: with a small, well-featured dataset, CRFs can shine (fast, convex training). With massive data, deep models usually outperform (as we will see).

Where an they be used in NLP

Use a CRF whenever you want to assign a label to each token, and the correctness of one label depends on its neighbors. Especially for:

- Named entity recognition
- POS tagging
- Chunking
- Slot filling
- Sequence labeling tasks using BIO/BILOU schemes

Why still relevant

CRFs remain useful because they are data-efficient, computationally efficient, and structurally constrained sequence labeling models.

Unlike LLMs, CRFs do not require large annotated datasets or GPUs and can run on low-power devices with very low latency.

They allow explicit incorporation of domain knowledge and enforce valid label transitions, producing deterministic and interpretable output.

Although LLMs are superior for complex contextual reasoning, CRFs are still the preferred choice in low-resource, low-power, or high-regulation environments.

Teaching note: Good exam-style takeaway: bigger/newer isn't always better — model choice depends on data availability, compute budget, and interpretability requirements.

Overview of Natural Language Processing methods

Linguistics

Morphology:

- Lexicon's
- Inflection & stemming rules + exceptions.

Syntax:

- Finite State Grammar (FSG). Only look past words.
- Context Free Grammar (CFG). Look at past and future words.
- *Add probability* with a PCFG by estimating probabilities from large corpora of natural language.

Semantics:

- Dependency Grammar: captures forwards and backwards semantic relations.

Statistics

Syntax or semantics:

- HMM.
- MEMM – *add linguistic information* with feature functions. Label bias problem.
- CRF – *add linguistic information* with feature functions.

Deep Learning

Syntax or semantics:

- LSTM (looks only at past words – Word2Vec & GloVe)
- Bi-Directions LSTM (looks at past and future words – Elmo)
- Transformers & BERT: multi-headed attention

From CRF to Deep Learning Models

- In the next lecture, we will take the step to deep learning models.

Questions?


THE CASE OF "RACE"
follows "to"
+0.8
Verb
+0.5
race
Noun
capital letter
-0.5
The Race
has suffix -ed
+1.2
previous word was "the"
+1.5
-0.5
-0.9
previous word was
has suffix -ed
+0.9
next word is "car"
CRF (Conditional Random Field) - Int ted Evidence
Hobbs algorithm
PREVIOUS WORD
NAIVE BAYES BAG

Exercise 1: HMM Parameter Estimation & Viterbi POS Disambiguation

Consider an HMM POS tagger evaluating the ambiguous word *"race"* in the sentence fragment: "Secretariat is expected to race tomorrow". The two candidate tags for *"race"* at position $t = 5$are Verb (VB) and Noun (NN). The preceding tag at $t = 4$for the word *"to"* was TOwith a path probability of $\delta_4(\text{TO}) = 0.42$. You are given the following simplified model parameters:

- Transition probabilities: $a_{\text{TO,VB}} = 0.80$, $a_{\text{TO,NN}} = 0.05$
- Emission probabilities: $b_{\text{VB}}(\text{race}) = 0.05$, $b_{\text{NN}}(\text{race}) = 0.06$

- Using the Viterbi recurrence $\delta_t(j) = b_j(O_t) \cdot \max_i\left[\delta_{t-1}(i) \cdot a_{ij}\right]$, calculate the path probabilities $\delta_5(\text{VB})$and $\delta_5(\text{NN})$.
- Notice that the emission probability of *"race"* as a noun (0.06) is actually higher than as a verb (0.05). Explain how the transition probabilities and syntactic context allow the Viterbi algorithm to correctly disambiguate *"race"* as VB.
- When scaling the Viterbi algorithm to long sentences with dozens of tags, explain why practical implementations convert products of probabilities into sums of log-probabilities ($\ell\delta_t, \ell a_{ij}, \ell b_j$). What hardware/floating-point problem does this prevent?

Exercise 2: MEMMs and the Label Bias Problem

Maximum Entropy Markov Models (MEMMs) replace fixed transition and emission lookup tables with a discriminative log-linear model conditioned on rich input features:

$$P(y_t \mid y_{t-1}, x) = \frac{\exp\left(\sum_k \theta_k f_k(x, y_{t-1}, y_t)\right)}{Z(x, y_{t-1})}$$

- In the slide example for tagging *"race"*, suppose the active feature weights $\sum \theta_k f_k$ evaluate to:
  - For VB: weights $0.8 + 0.01 + 0.1 = 0.91$
  - For NN: weights $0.8 - 1.3 = -0.5$
  - Using the softmax equation above, calculate the normalized local probabilities $P(\text{VB} \mid x)$ and $P(\text{NN} \mid x)$.

- Explain the **Label Bias Problem** inherent in MEMMs. Why does local normalization at each time step cause states with few outgoing transitions to dominate over states with many outgoing transitions, regardless of what the observation evidence supports?

- Provide an intuitive real-world analogy (e.g., navigating street corners or road networks) explaining how local step-by-step normalization differs from global sequence optimization.

Exercise 3: Generative vs. Discriminative Sequence Modeling (HMM →MEMM →CRF)

Review the theoretical evolution from generative HMMs to discriminative CRFs.

- Contrast HMMs and CRFs across the following architectural dimensions:
  - Model type: Which models the joint distribution $P(X, Y)$and which models the conditional distribution $P(Y \mid X)$?
  - Input features: Why are HMMs strictly limited to discrete word tokens, whereas CRFs can incorporate arbitrary overlapping feature functions (e.g., capitalization, suffixes like -ing, regex digit patterns)?

- How does a Linear-Chain CRF mathematically eliminate the label bias problem found in MEMMs? (Hint: Address where the partition function $Z(x)$normalizes the scores.)

- Despite the dominance of modern deep transformers (such as BERT or LLMs), list two engineering or practical reasons why CRFs remain the preferred choice for sequence labeling tasks in low-power, low-resource, or highly regulated production environments.

Discussion Lecture: Statistical NLP



Maastricht University

On the Markov Assumption: Can a Wrong Model Be the Right Tool?

A first-order Markov model assumes that the next state depends only on the current one. Yet subject and verb agree across whole clauses (“The captain of the three stranded fishing boats is arriving”), and a question is formed by moving a word to the front because of something at the end.

Does language have a Markovian memory, or are we imposing a memoryless chain on a hierarchical process because the chain is what we can compute?

HMMs were known to be wrong about language and were still the industrial standard for two decades. If a model can be false and still the best available tool, what is the relation between the truth of a model and its usefulness?

On Generative vs. Discriminative: Is Explaining the Input a Form of Understanding?

A generative model (HMM) tells a story about how the words came to be: tags produce words. A discriminative model (CRF) tells no story; it is a decision rule from words to tags. For tagging, the decision rule won.

Then the most successful language models in history turned out to be generative again: they do nothing but predict the next word. What changed: the data, the compute, or our idea of what a model is for?

Does a model that can explain how its input arose know more about language than one that only classifies it? Or is “explaining the input” just a longer way of memorising it?

On Commitment: Should a Reader Decide Now, or Wait for the End?

The label bias problem is, underneath the mathematics, a question about commitment: a model that normalises locally has to decide at every step and can never take a decision back. A globally normalised model defers every decision until it has seen the whole sentence.

Humans read incrementally and still get garden-path sentences wrong (“The horse raced past the barn fell”), then repair. Is understanding a local process that occasionally backtracks, or a global one that we only experience as local?

Transformers look at the whole sentence at once. Did they solve the problem of premature commitment, or did they abolish the idea that reading has an order at all, and what is lost if it does not?

References

- More information on most statistical methods can be found here:
- Speech and Language Processing (Dan Jurafsky and James H. Martin). Prentice Hall, 2021. https://web.stanford.edu/~jurafsky/slp3/

- Additional information on Conditional Random Fields (CRF): Sutton, C & McCallum, A 2012, 'An Introduction to Conditional Random Fields', Foundations and Trends in Machine Learning, vol. 4, no. 4, pp. 267-373. https://doi.org/10.1561/2200000013